

Deep Learning

Irene Siragusa, PhD

Outline

-  Introduction
-  Multi-Layer Perceptron
-  Architecture
-  Training
-  Regularization
-  Optimization
-  Principal Deep Neural Networks

Introduction

- Deep Learning (DL) was created to provide a training framework for algorithms that have good generalization capacity in the most challenging tasks of Artificial Intelligence such as Computer Vision, Speech Recognition, Natural Language Processing
- These tasks cannot, in fact, be solved with traditional ML algorithms or with classical, shallow neural networks

Introduction

- The most relevant problems from a computational point of view are:
 - Inability to represent highly complex functions to map inputs to desired outputs
 - Curse of dimensionality
 - Relevant configurations of the inputs grow exponentially with the dimensions and therefore can be many more than the training examples
 - Metrics or distances calculations between samples during training

Introduction

- The need to compute complex functions on the input data arises from the fact that the generalization capacity of an ML algorithm depends on priors
- Regularization terms expressed as smoothing conditions, can be explicitly included in loss functions to overcome this issue, i.e. probabilistic prior

Introduction

- Essentially these priors tend to favor a function that remains unchanged in a small neighbour of a given data point

$$f(\mathbf{x}) \approx f(\mathbf{x} + \epsilon)$$

- The generalization capacity is then limited
 - A lot of well distributed samples in zone in which f highly varies are needed
 - As dimensions increase, priors not guarantee smoothness along all the directions

We need algorithms able to learn more complex functions from (relative) few data points

Introduction

- DL manages complexity by making some assumptions about the hypothesis space:
 - Data are generated by the composition of many simple factors called features, aggregated according to different levels of a hierarchy
 - The perception of a face is based on the perception of an ovoid shape and a certain skin pigment, but also of some specific characteristics such as the presence of eyes, nose and mouth, and, subsequently, of the shape and color of each feature with its variants

Introduction

- DL manages complexity by making some assumptions about the hypothesis space:
 - Not all input data configurations are relevant to the problem
 - f can assume relevant values only in a manifold, over a connected subset of points in $\mathbb{R}^d$ which has an intrinsic topology with a lower dimensionality (manifold hypothesis)
 - A manifold topological variety, such as a 2D surface or a 1D curve in $\mathbb{R}^3$

Neural Network

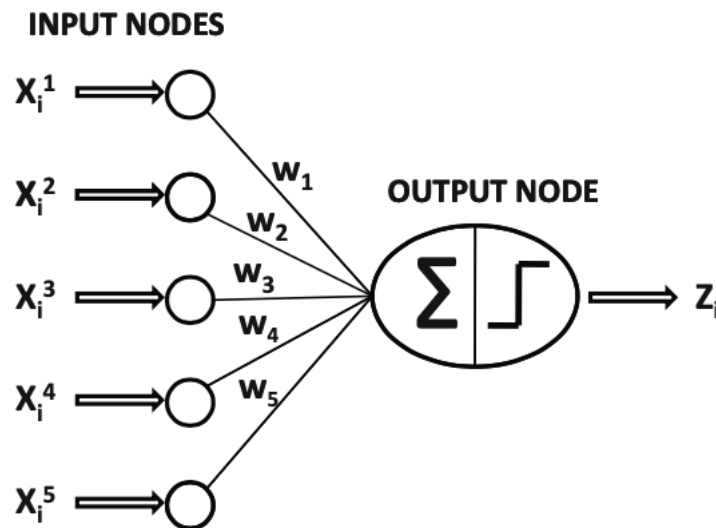
- Neural Networks (NN) are a model of simulation of the human nervous system.
 - The human nervous system is composed of cells, called neurons, which can be connected with each other, thus making, synapses. Learning process modifies the strength of synaptic connections between neurons, in response to external stimuli.
- Individual nodes in NN are referred to as neurons, which are units of computation that receive input from some other neurons, make computations on these inputs, and feed them into yet other neurons.
- The activation function is defined by the weights on the input connections to that neuron, which can be viewed as analogous to the strength of a synaptic connection.

Neural Network

- By changing these weights appropriately, the computation function can be learned, which is analogous to the learning of the synaptic strength in biological neural networks.
- The “external stimulus” in artificial neural networks for learning these weights is provided by the training data.
 - The idea is to incrementally modify the weights whenever incorrect predictions are made by the current set of weights.
- The key to the effectiveness of the neural network is the architecture used to arrange the connections among nodes.

Single Layer perceptron

- The most simple NN is made up of the perceptron, which is a single computational unit (neuron)
- One input node for each feature, connected with a weighted connection to the single one output node



Algorithm *Perceptron*(Training Data: $\mathcal{D}$)

begin

Initialize weight vector $\bar{W}$ to random values;

repeat

Receive next training tuple $(\bar{X}_i, y_i)$;

$z_i = \bar{W} \cdot \bar{X}_i + b$;

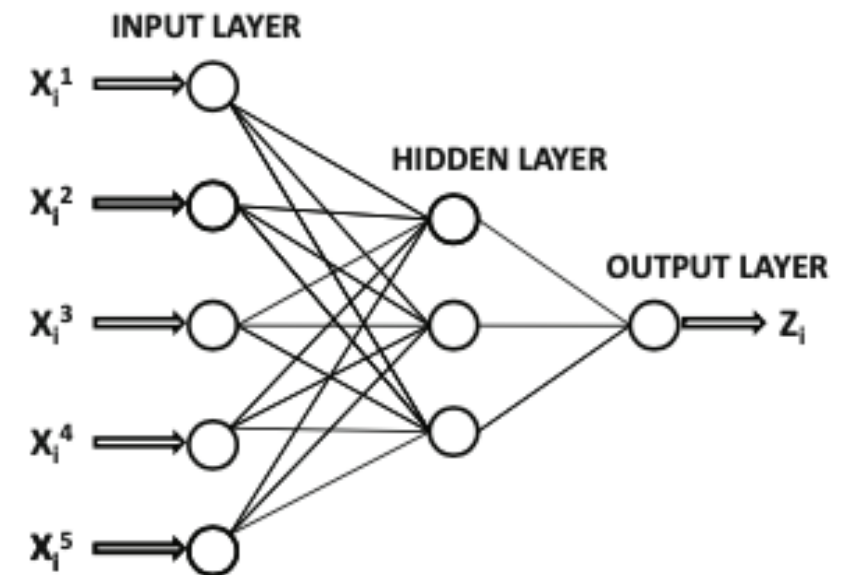
$\bar{W} = \bar{W} + \eta(y_i - z_i)\bar{X}_i$;

until convergence;

end

Multi-Layer Perceptron

- Deep Neural Networks (DNN) are based on the Multi-Layer Perceptron (MLP)
- MLP adds an hidden layer to the perceptron input and output layers
- Those networks are *dense* or fully connected (multilayer feed-forward network) that is *each* neuron of a given layer, has as input the output of *all* the neurons of the previous layer



Multi-Layer Perceptron

- MLP learns a classification leveraging the input data

$$y = f^*(x)$$

- Network architecture is given by a series of n layers, where m computational units are placed.
- n is the depth of the network, m is its width
- Internal layers are referred as hidden layers and its activation function is referred to as h
- The network approximates f^* as $f(\mathbf{x};\theta)$ and it is trained to find the optimal parameters θ for the better approximation of f^*

Multi-Layer Perceptron

- $f(\mathbf{x};\theta)$ is calculated concatenating the contributions of each activation function at each layer

$$f(\mathbf{x}) = f^{(n)}(\dots f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x}))))$$

- f is defined as a linear operator applied to a non-linear mapping $\phi(\mathbf{x})$ of inputs

$$\begin{aligned} y = f(\mathbf{x}; \theta, \mathbf{w}) &= \phi(\mathbf{x}; \theta)^\top \mathbf{w} + b_o = \\ &= h_o(h_1(\mathbf{x}; \gamma)^\top \mathbf{w}_1 + b_1)^\top \mathbf{w} + b_o = \\ &= h_o(h_1(\dots (h_{in}(\mathbf{x})^\top \mathbf{w}_{in} + b_{in}) \dots)^\top \mathbf{w}_1 + b_1)^\top \mathbf{w} + b_o \end{aligned}$$

- Objective of the network is also to learn the *optimal representation* for ϕ

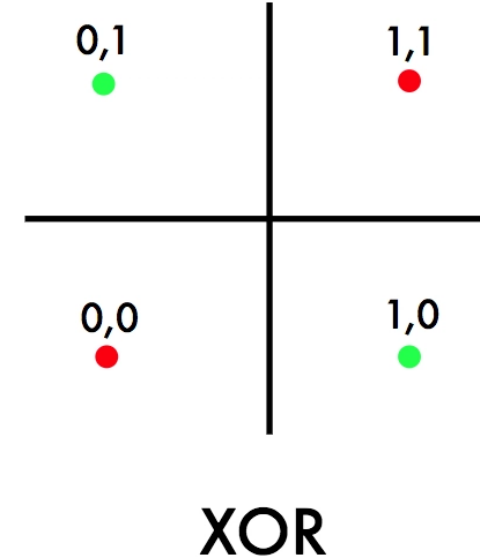
Architecture

- The architecture of a DNN is very complex and is isomorphic to a graph, referred to as computational graph, which has the following roles
 - Calculation of forward activations *through* the network
 - Calculating gradients and *backward* errors in backpropagation training
- DL frameworks explicitly rely on a graph compiling component to map operations in hardware

Architecture

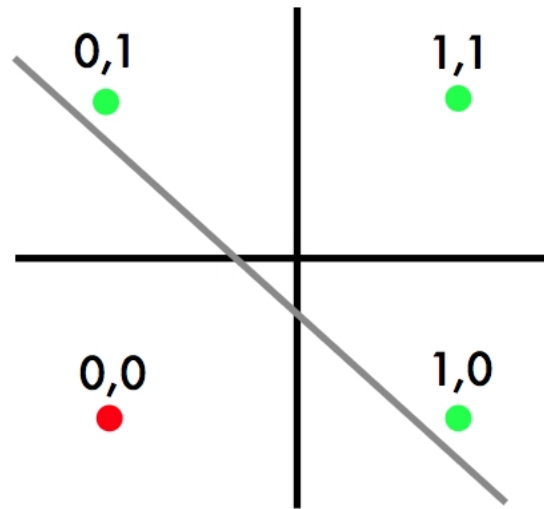
- e.g. XOR problem
 - The XOR operation is a binary operation that takes two binary inputs and produces a binary output. The output of the operation is 1 only when the inputs are different.

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	0

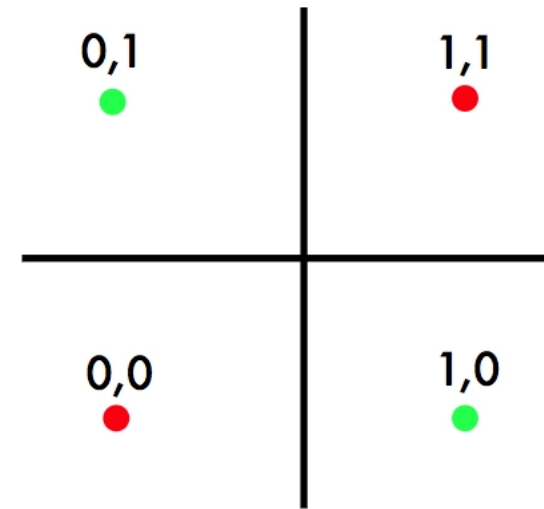


- Can we train a ML model to perform XOR operation?

Architecture



OR



XOR

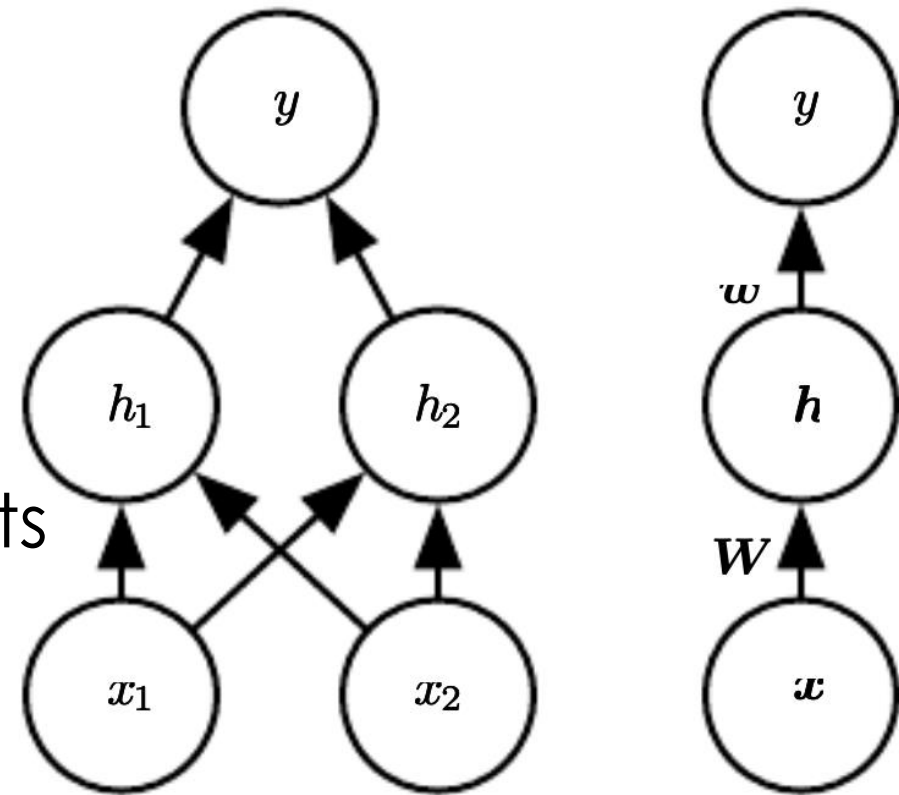
Architecture

- e.g. Network for XOR

$$y = \mathbf{h}^\top \mathbf{w} + b$$

$$\mathbf{h} = \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\}$$

In h a non-linear function ReLU is applied, and each column of $\mathbf{W}$ represents the vector of weights $\mathbf{W}_i$ connecting the inputs to the h_i hidden unit

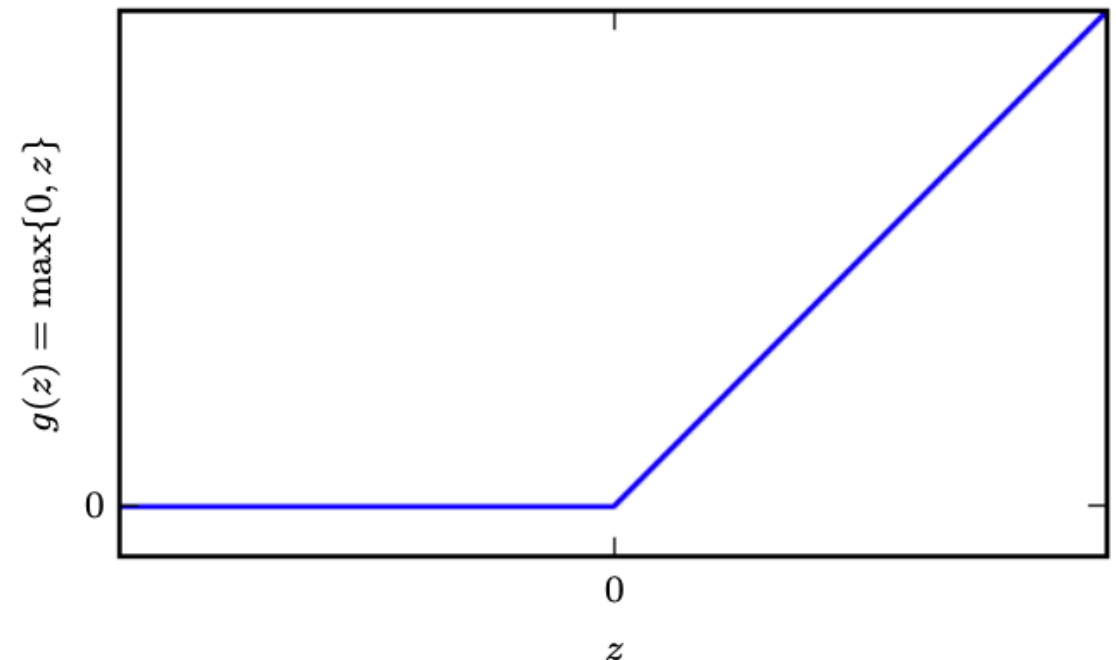


Architecture

- ReLU function, thanks to its non-linearity, can solve the xor problem, since it operates at hidden layer level

$$\text{ReLU}(x) = \max(0, x)$$

In addition, for binary classification purposes, a single unit is needed, but it classifies over the output of activation functions and not on native data



Architecture

- The choice of the number of output units, activation and hidden functions depends on a few factors
 - Allow the estimation of complex nonlinear functions
 - Satisfy the functional form of an MLE or MAP estimator with respect to the probability distribution of the values of the outputs
 - Have a large and well estimable gradient to support learning algorithms that rely on descending along the gradient to minimize the error made on the outputs (loss)

Architecture

- The possibilities for the loss function are the same as ML models
 - Cross-entropy loss provides MLE estimation of a conditional probability distribution $p(\mathbf{y} \mid \mathbf{x})$

$$\mathcal{L}(\boldsymbol{\theta}) = -\mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{y} \mid \mathbf{x})$$

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2} \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \hat{p}_{\text{data}}} \|\mathbf{y} - f(\mathbf{x}; \boldsymbol{\theta})\|^2 + \text{const} \quad \text{MSE}$$

Architecture

- We can be interested towards a regression task for estimate a $f^*(\mathbf{x};\boldsymbol{\theta})$ over $\mathbf{y}$
- Thus the learning task is to estimate the optimal f^*

$$f^* = \arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}} \|\mathbf{y} - f(\mathbf{x})\|^2$$

Mean Squared Error estimates the mean of $\mathbf{y}$

$$f^* = \arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}} \|\mathbf{y} - f(\mathbf{x})\|_1$$

Mean Absolute Error estimates the median of $\mathbf{y}$

Architecture

- Let assume that the estimate is learned from the hidden layers in terms of the parameters, so we will have that $\mathbf{h} = f(\mathbf{x}; \theta)$
- Linear output units can be used for estimate the mean of $\mathbf{y}$, assuming a Gaussian distribution and using MSE

$$\hat{\mathbf{y}} = \mathbf{W}^\top \mathbf{h} + \mathbf{b}$$

$$p(\mathbf{y}|\mathbf{x}) = \mathcal{N}(\mathbf{y}; \hat{\mathbf{y}}, \mathbf{I})$$

Architecture

- As in a binary classification, a single output unit will provide the MLE of $P(y=1 \mid \mathbf{x}) \in [0,1]$ which is a Bernoulli distribution
- We will assume that the input to the unit is always linear

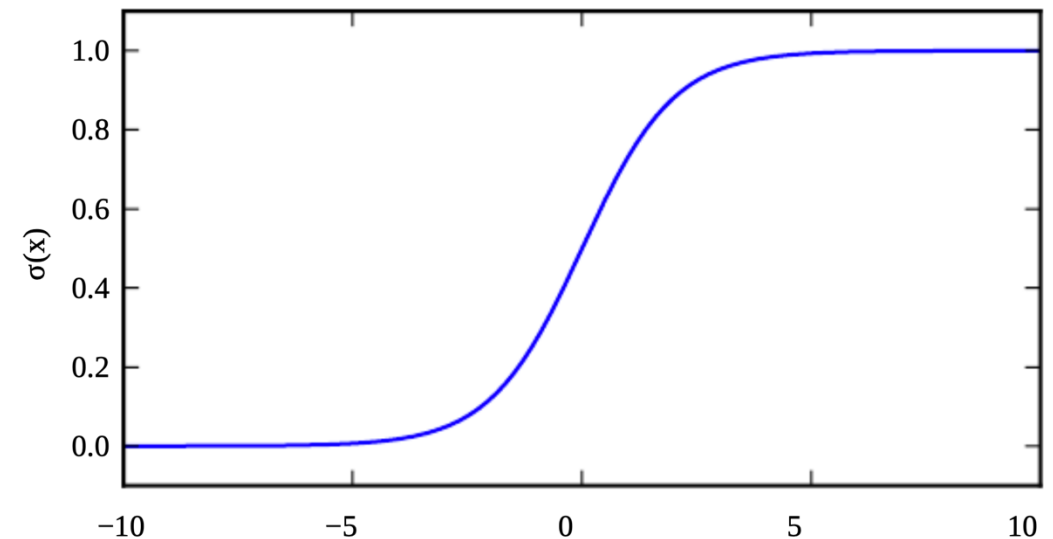
$$z = \mathbf{W}^\top \mathbf{h} + b$$

- z will provide the estimate of the log-probability of y not normalized, called logit, which is the real output of hidden layers of the NN

Architecture

- A normalization function is needed since probability must be in $[0,1]$
- A linear activation with threshold in 0 and 1 is not suitable because the gradient cancels out as soon as $z = \text{logit}(P) \notin [0,1]$
- A sigmoidal function can be used (sigmoid)

$$\sigma(x) = \frac{1}{1 + \exp(-x)} \equiv \frac{\exp(x)}{\exp(x) + \exp(0)}$$



Architecture

- Assuming that the non normalized log-probability is linear in y and z :

$$\log \tilde{P}(y) = yz$$

$$\tilde{P}(y) = \exp(yz)$$

the sigmoid provides the estimation of $y=1$, that is $P(y)$

$$P(y) = \frac{\exp(yz)}{\sum_{y'=0}^1 \exp(y'z)} = \frac{\exp(yz)}{\exp(0 \cdot z) + \exp(1 \cdot z)}$$

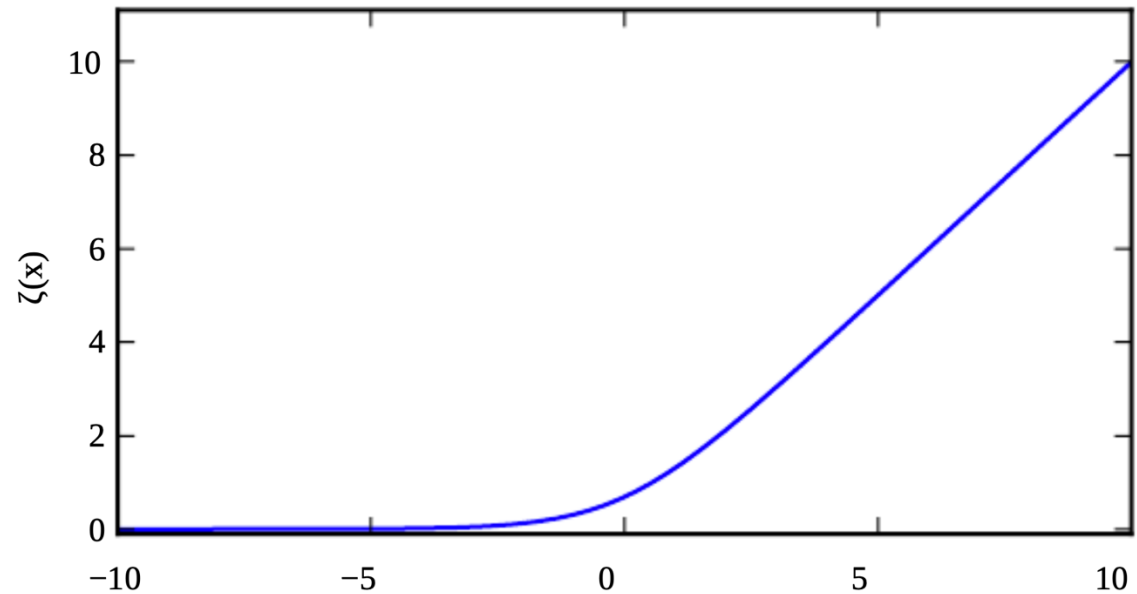
$$P(y \equiv 1) = \frac{\exp(z)}{1 + \exp(z)} = \sigma(z) = \sigma(\mathbf{W}^\top \mathbf{h} + \mathbf{b})$$

Architecture

- We obtain the loss function corresponding to MLE learning

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= -\log P(y \mid \boldsymbol{x}) \\ &= -\log \sigma(z) \\ &= \log(1 + \exp(z)) \\ &= \zeta(z)\end{aligned}$$

softplus



The logarithm of the exponential provides a linear function which has good behavior for the gradient

Architecture

- As in a multiclass classification, the objective is to estimate a Multinoulli distribution

$$\hat{\mathbf{y}} = \{\hat{y}_i = P(y = i|\mathbf{x})\}$$

$$\sum_i \hat{y}_i = 1$$

- Concepts drawn for the sigmoid, can be extended towards the softmax function

Architecture

- The softmax function, using a loss for MLE learning, satisfies the requirements to have sum 1 on all components and also those related to the calculation of the gradient

$$\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b} = \{z_i = \log \tilde{P}(y = i | \mathbf{x})\}$$

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Since the estimated probabilities are from time to time, concentrated on a single class i , and z_j is very small if different from $\max_j z_j$

$$\log \text{softmax}(z_i) = z_i - \log \sum_j \exp(z_j) \approx z_i - \max_j z_j$$

Architecture

- The sigmoid activation function provides the MLE estimate of a binomial distribution by minimising the cross-entropy
- The softmax activation function provides the MLE estimate of a multinomial distribution by minimising the cross-entropy

This is how we connects DNN with statistics

Architecture

- Binary classification (2 classes):
 - Single output unit w/ sigmoid function
 - Binary cross-entropy loss or similar (e.g. KL divergence)
- Multi-class (single label) classification (N classes):
 - N output units with softmax function
 - Predicted class is calculated as
$$y_c = \arg \max_{i \in 1, \dots, N} \text{softmax}(\mathbf{z}), \quad \mathbf{z} = \{z_1, z_2, \dots, z_N\}$$
 - Multi-class cross-entropy loss or similar

Architecture

- Multi-class multi-label classification (N classes):
 - A given sample can be associated with different labels at the same time
 - N different outputs units are used with sigmoid function
 - Each output unit is trained with binary cross-entropy or similar
- NOTE
 - A custom loss for the given problem can be designed, *but* if the task is a classification, one of the previous seen losses is the best choice.
 - Custom components can be then added if needed

Architecture

- Activation of the hidden units
 - Also in this case the network will apply an activation function $g(.)$ to a similar transformation on its inputs

$$\mathbf{h} = g(\mathbf{W}^\top \mathbf{x} + \mathbf{b})$$

- Generally, $\mathbf{b}$ is initialized in each layer with small values, e.g. 0.1
- Rectified Linear Unit (ReLU) assumes $g(z) = \max(0, z)$

Architecture

- ReLU and its variants
 - ReLU is great for building nonlinearities through the mechanism of layer-by-layer composition, *but* it cannot be differentiable for $z=0$
 - From a numerical point of view, in $z=0$ left or right derivative or $g(.)$ is used, which both are defined
 - There is always a numerical error in the exact calculation of a value of z , so it can always be assumed that $g(0)$ actually corresponds to a value $g(0 + \varepsilon)$ with ε very small and for which well-defined gradient exists

Architecture

- ReLU and its variants

$$h_i = g(\mathbf{z}, \boldsymbol{\alpha})_i = \max(0, z_i) + \alpha_i \min(0, z_i)$$

- ReLU: $\alpha_i = 0$
- Linear unit : $\alpha_i = 1$
- Leaky ReLU (LReLU): α_i very small (e.g. 0.01)
- Parametric ReLU (PReLU): α_i is a learnable parameter

Architecture

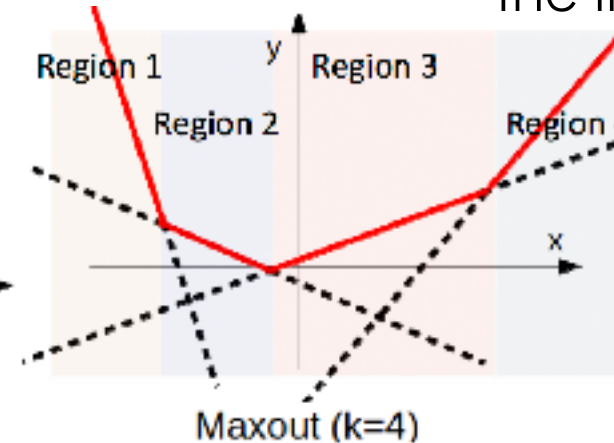
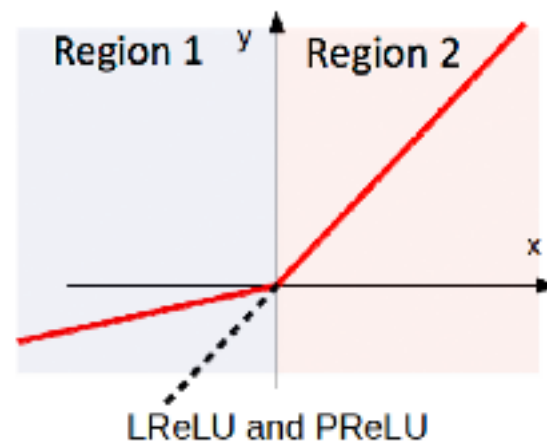
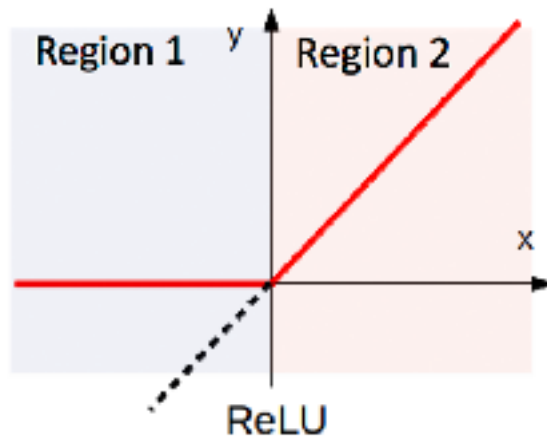
- Maxout

$$g(\mathbf{z})_i = \max_{j \in \mathbb{G}^{(i)}} z_j = \max_{j \in \mathbb{G}^{(i)}} (\mathbf{W}_{:,j}^\top \mathbf{x} + b_j)$$

i-th component of the output vector is the maximum of k linear combinations of the inputs

$$\mathbb{G}^{(i)} = \{(i-1)k + 1, \dots, ik\}$$

The unit then learns a piecewise linear function of the inputs on k regions



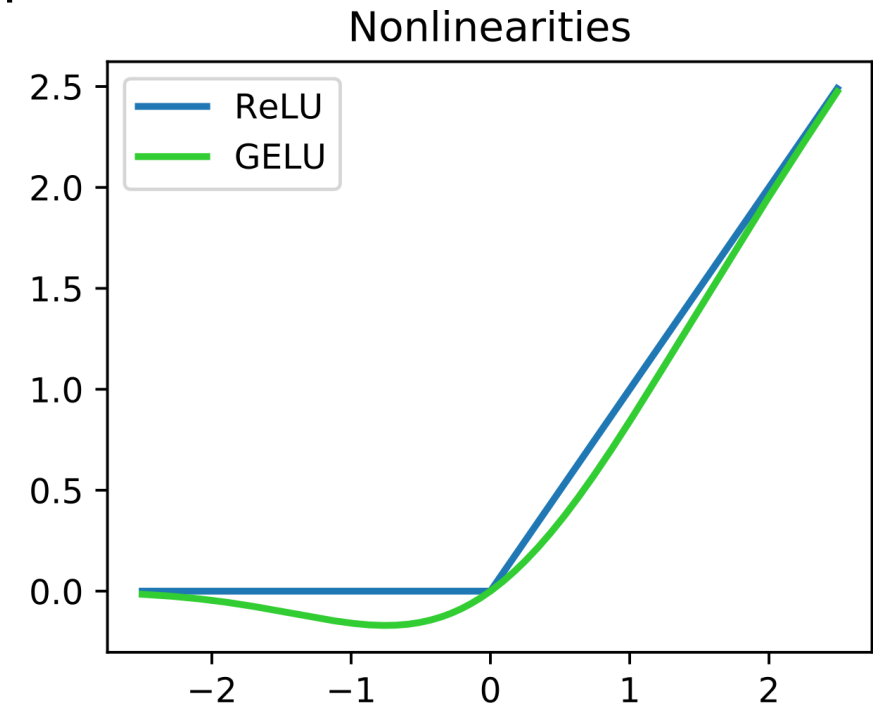
Architecture

- Gaussian Error Linear Unit (GeLU) is a variant of ReLU in which the cumulative function of the Normal distribution is used to modulate linear activation

$$\text{GELU}(x) = xP(X \leq x)$$

$$X \sim \mathcal{N}(0, 1)$$

$$\text{GELU}(x) \sim x\sigma(1.702x)$$



Architecture

- Other activation functions for hidden units
 - Sigmoid
 - Hyperbolic tangent
 - Softplus
 - Radial Basis Function (RBF)

Architecture

- The Universal Approximation Theorem states that
- A feedforward network with a linear output layer and at least one hidden layer with any *squashing* activation function (s.a. the logistic sigmoid activation function) can approximate any Borel measurable function from one finite-dimensional space to another with any desired nonzero amount of error, provided that the network is given enough hidden units.
 - Every continuous function in any closed and limited subset of $\mathbb{R}^n$ is Borel-measurable
- The derivatives of the feedforward network can also approximate the derivatives of the function arbitrarily well

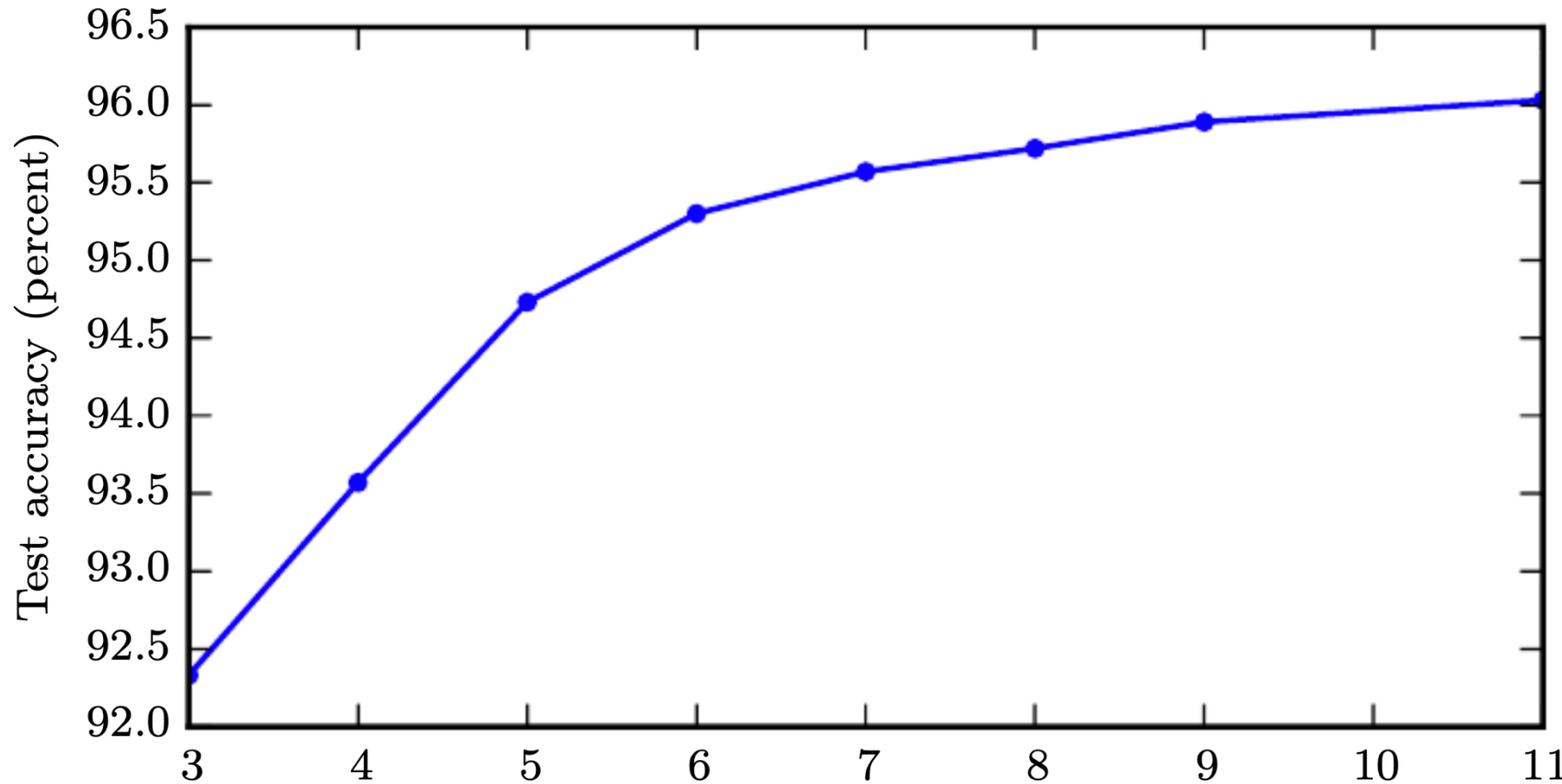
Architecture

- Obviously such a network would be enormously large, in width
- The solution has been to go deep, using multiple hidden layers
 - It is assumed that a complex function can be approximated as a composition of linear trait functions
 - The pattern we need to predict depends on a set of *complex features* that, in turn, depend on other *simpler features*, and so on
- Obviously, the problem is moved toward an emergent parameter number problem that explodes exponentially

Architecture

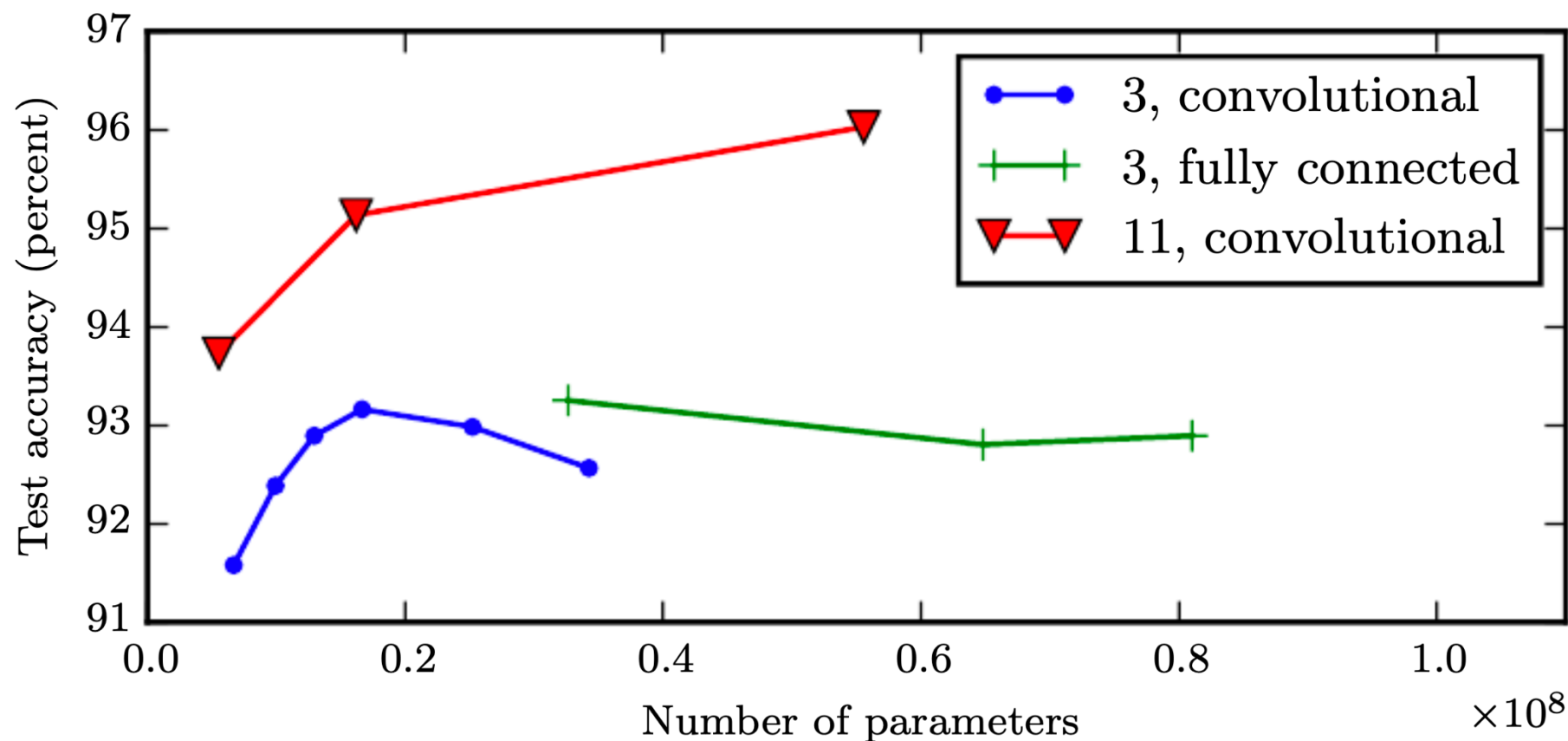
- Hidden layers do not need to be dense
 - In Convolutional Neural Networks (CNN) each unit of $i+1$ layer, is linked only to the single units in layer i which compose the support for the convolutional kernel, used for computing the inputs for the given unit
 - A significant reduction in terms of parameters, is also associated with an efficiency increase with deeper networks which can be built thanks to the reduced number of connections

Architecture



Accuracy increment as increasing the number of hidden layers over a data set of house number images, acquired from Google Street View

Architecture



Decreasing of the number of parameters as increasing the depth of the network, over a data set of house number images, acquired from Google Street View

Training

- Training algorithm of a DNN, as in the simple case of the MLP, is done via an algorithm of gradient descend over a cost or loss function $J(\boldsymbol{\theta}, \mathbf{x})$ with respect to the parameters of the models
- In particular, the so called Stochastic Gradient Descent algorithm is used
 - $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}, \mathbf{x})$ will be calculated to minimize J

Training

- Calculus of $\nabla_{\theta} \mathcal{L}(\theta, \mathbf{x})$ involves all parameters, also the ones in the more hidden and deep layers
- An algorithm is needed to recursively calculate the gradients thus building and using a computational graph. This algorithm is the Backpropagation

Training

- Backpropagation

- It is based on the calculus of the gradient of a composite function $z = f(g(\mathbf{x}))$
- It can be extended in the case of functions of n -dimensional tensors $\mathbf{x}$

$$\mathbf{y} = g(\mathbf{x}) \text{ and } z = f(\mathbf{y})$$

$$\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^{\top} \nabla_{\mathbf{y}} z,$$

$$\nabla_{\mathbf{x}} z = \sum_j (\nabla_{\mathbf{x}} Y_j) \frac{\partial z}{\partial Y_j}$$

Training

- Backpropagation

Require: Network depth, l

Require: $\mathbf{W}^{(i)}, i \in \{1, \dots, l\}$, the weight matrices of the model

Require: $\mathbf{b}^{(i)}, i \in \{1, \dots, l\}$, the bias parameters of the model

Require: $\mathbf{x}$, the input to process

Require: $\mathbf{y}$, the target output

$$\mathbf{h}^{(0)} = \mathbf{x}$$

for $k = 1, \dots, l$ **do**

$$\mathbf{a}^{(k)} = \mathbf{b}^{(k)} + \mathbf{W}^{(k)} \mathbf{h}^{(k-1)}$$

$$\mathbf{h}^{(k)} = f(\mathbf{a}^{(k)})$$

end for

$$\hat{\mathbf{y}} = \mathbf{h}^{(l)}$$

$$J = L(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \Omega(\theta)$$

Forward and prediction

Loss computation

Training

- Backpropagation

After the forward computation, compute the gradient on the output layer:

$$\mathbf{g} \leftarrow \nabla_{\hat{\mathbf{y}}} J = \nabla_{\hat{\mathbf{y}}} L(\hat{\mathbf{y}}, \mathbf{y})$$

for $k = l, l - 1, \dots, 1$ **do**

Convert the gradient on the layer's output into a gradient on the pre-nonlinearity activation (element-wise multiplication if f is element-wise):

$$\mathbf{g} \leftarrow \nabla_{\mathbf{a}^{(k)}} J = \mathbf{g} \odot f'(\mathbf{a}^{(k)})$$

Compute gradients on weights and biases (including the regularization term, where needed):

$$\nabla_{\mathbf{b}^{(k)}} J = \mathbf{g} + \lambda \nabla_{\mathbf{b}^{(k)}} \Omega(\theta)$$

$$\nabla_{\mathbf{W}^{(k)}} J = \mathbf{g} \mathbf{h}^{(k-1)\top} + \lambda \nabla_{\mathbf{W}^{(k)}} \Omega(\theta)$$

Propagate the gradients w.r.t. the next lower-level hidden layer's activations:

$$\mathbf{g} \leftarrow \nabla_{\mathbf{h}^{(k-1)}} J = \mathbf{W}^{(k)\top} \mathbf{g}$$

end for

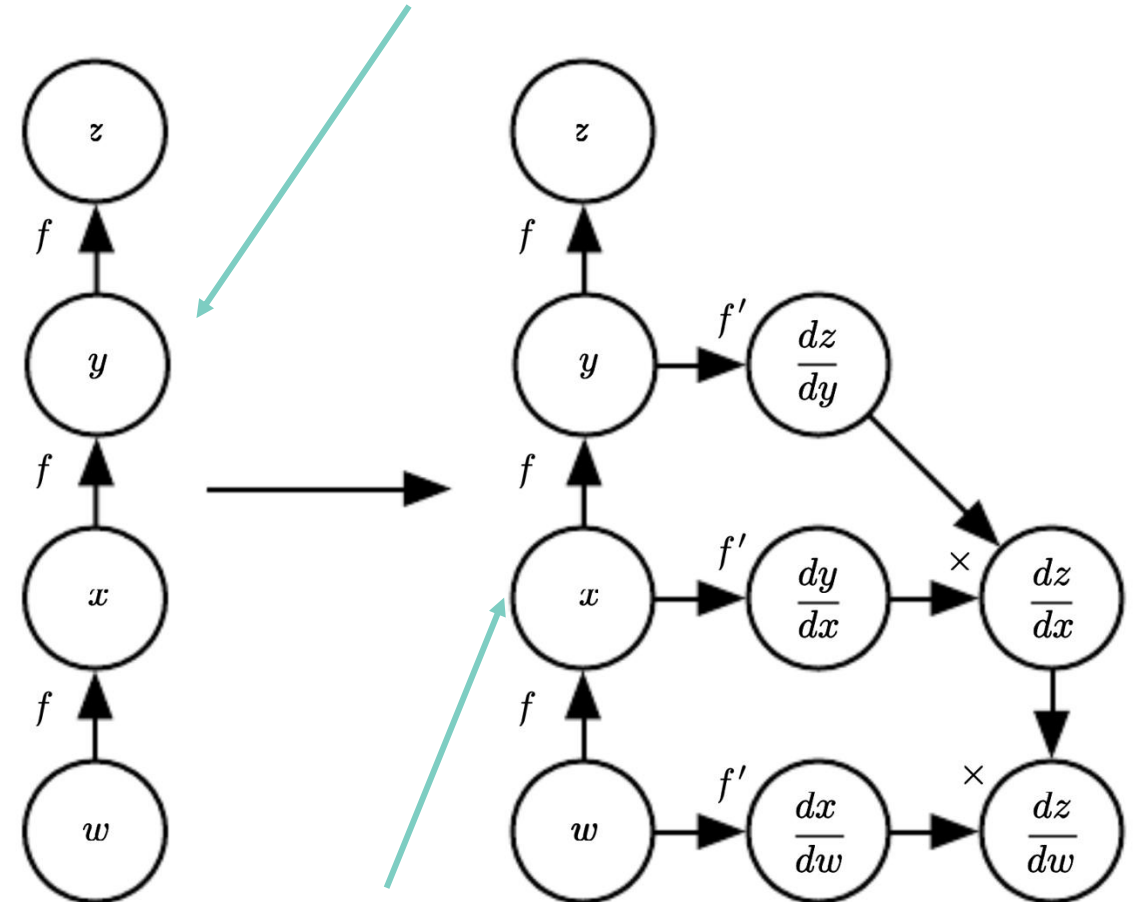
Training

- Backpropagation

- The calculation of compound derivatives can be carried out through the computational graph

$$\begin{aligned} & \frac{\partial z}{\partial w} \\ &= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w} \\ &= f'(y) f'(x) f'(w) \\ &= f'(f(f(w))) f'(f(w)) f'(w) \end{aligned}$$

PyTorch uses a symbol-to-number differentiation
Inputs $\mathbf{x}$ flow into the graph and gradually calculate the derivatives



Tensorflow uses a symbol-to-symbol differentiation
Inputs $\mathbf{x}$ inputs flow into a symbolic computation graph of activations and derivatives

Training

- Stochastic Gradient Descent (SGD)

- The Gradient Descent algorithm states that a function $f(\mathbf{x})$ can be minimized moving $\mathbf{x}$ along the direction of the gradient of f

$$\mathbf{x} \leftarrow \mathbf{x} - \varepsilon \nabla_{\mathbf{x}} f$$

- Gradient of $\mathcal{L}(\boldsymbol{\theta})$ is the mean of all gradient values for all the m inputs
 - The computational cost is $O(m)$ which can become very high

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Training

- Stochastic Gradient Descent (SGD)

- SGD samples m' inputs which forms a minibatch $\mathbb{B}$ of dozens or hundreds samples, over which the gradient is calculated and updates of weights is performed accordingly
- On doing this, the computational connection with number of samples is deleted

$$\mathbf{g} = \frac{1}{m'} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}) \quad \mathbb{B} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m')}\}$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \mathbf{g}$$

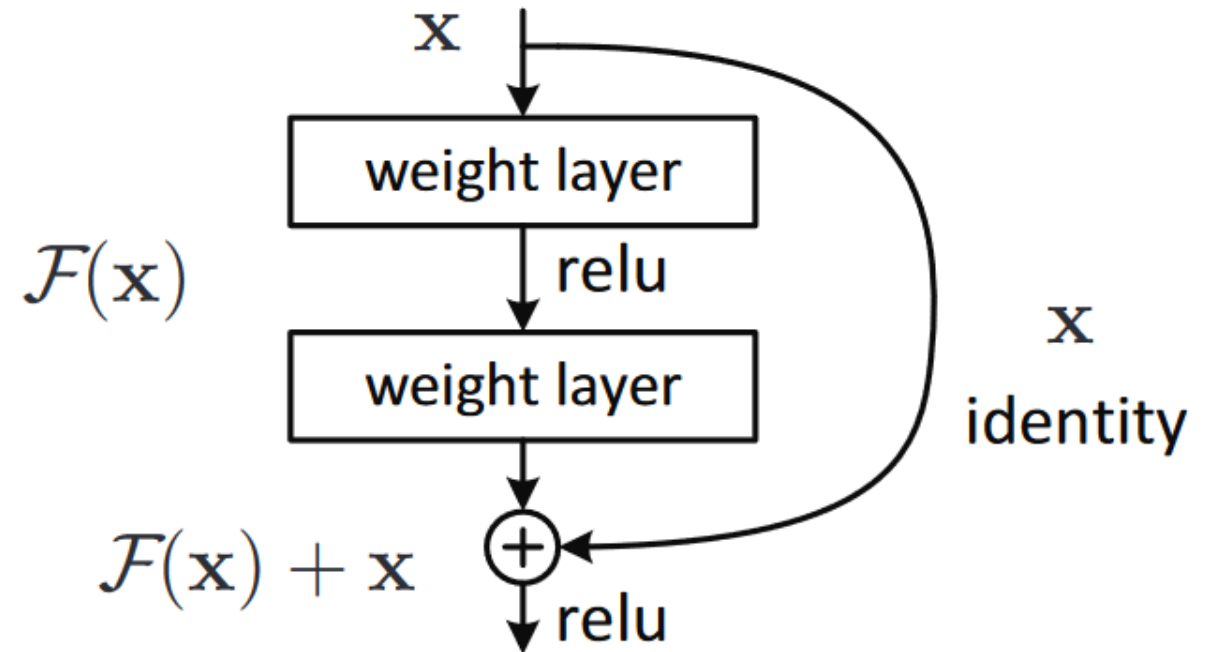
- We will refer as epoch a single presentation of the entire data set to the training algorithm

Training

- Backpropagation is based on a calculation of gradients at each layer which must remain *numerically stable*
- On the contrary, if this does not happen, a vanishing gradient situation occur, in which the derivative $\partial \mathcal{F} / \partial \mathbf{x}$ of a layer becomes very small and therefore poorly conditioned

Training

- Skip connections
 - A layer is forward connected with a subsequent layer, skipping the immediate next layer
 - This block is also called residual block
 - They may be used to overcome the vanishing gradient problem



Gradient from the residual block adds a constant term to bad-conditioned $\partial \mathcal{F} / \partial \mathbf{x}$ towards forcing the learning of $\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}$

Training

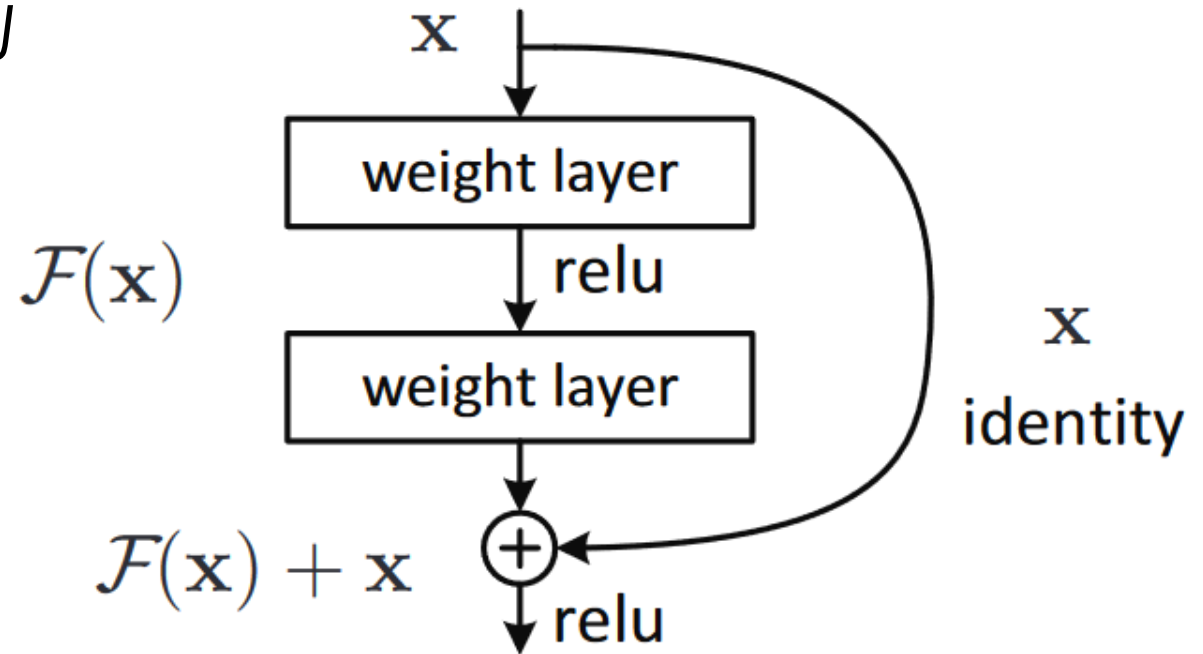
- The original derivative of loss J for $\mathbf{x}$ is:

$$\frac{\partial J}{\partial \mathbf{x}} = \frac{\partial J}{\partial \mathcal{F}(\mathbf{x})} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}}$$

- With adding skip connections

$$\frac{\partial J}{\partial \mathbf{x}} = \frac{\partial J}{\partial \mathcal{H}(\mathbf{x})} \left(\frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + 1 \right)$$

$$= \frac{\partial J}{\partial \mathcal{F}(\mathbf{x})} \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + \frac{\partial J}{\partial \mathcal{H}(\mathbf{x})}$$



This term ensures computational stability

Regularization

- Regularization techniques aim to reduce generalization error, and not the training one
- They include, among the others:
 - Usage of norms as regularization terms
 - Data augmentation
 - Multi-task learning
 - Early stopping
 - Dropout

Regularization

- Usage of L^2 norm

- Regularization term not involves bias, but only weights
- It is a quadratic term (such as the weight decay)
- Regularization over bias lead to underfitting

$$\tilde{\mathcal{L}}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \frac{\alpha}{2} \mathbf{w}^\top \mathbf{w} + \mathcal{L}(\mathbf{w}; \mathbf{X}, \mathbf{y})$$

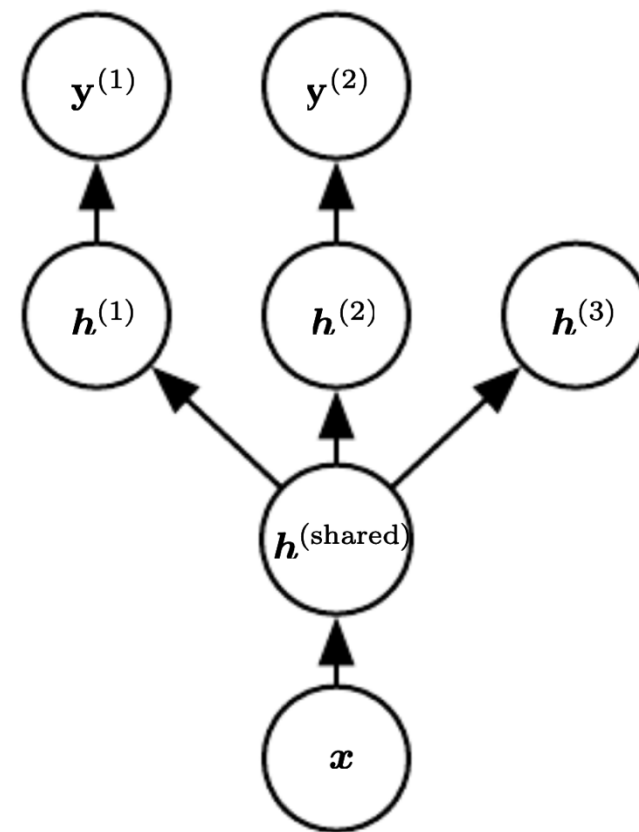
$$\mathbf{w} \leftarrow \mathbf{w} - \epsilon (\alpha \mathbf{w} + \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}; \mathbf{X}, \mathbf{y}))$$

Regularization

- Data augmentation
 - The number of samples is artificially increased to allow for better generalization
 - Data transformations that can be induced by the application domain can be used (e.g. affine transformations on images, SMOTE, ...)
 - Noise can be injected into the input data (it has the effect of L^2 regularizing) or even in the weights of hidden layers

Regularization

- Multi-task learning
 - It is also referred to transfer learning
 - Sharing the network to train it on different more or less similar tasks
 - It's like we have a multi-head network
 - Different transfer learning techniques



Regularization

- Transductive transfer learning
 - An already-trained network, referred to as a pre-trained backbone, is further trained on a selected domain of interest (domain adaptation)
 - Large Language Models, U-net
- Inductive transfer learning
 - Training domain is kept but the task changes

Regularization

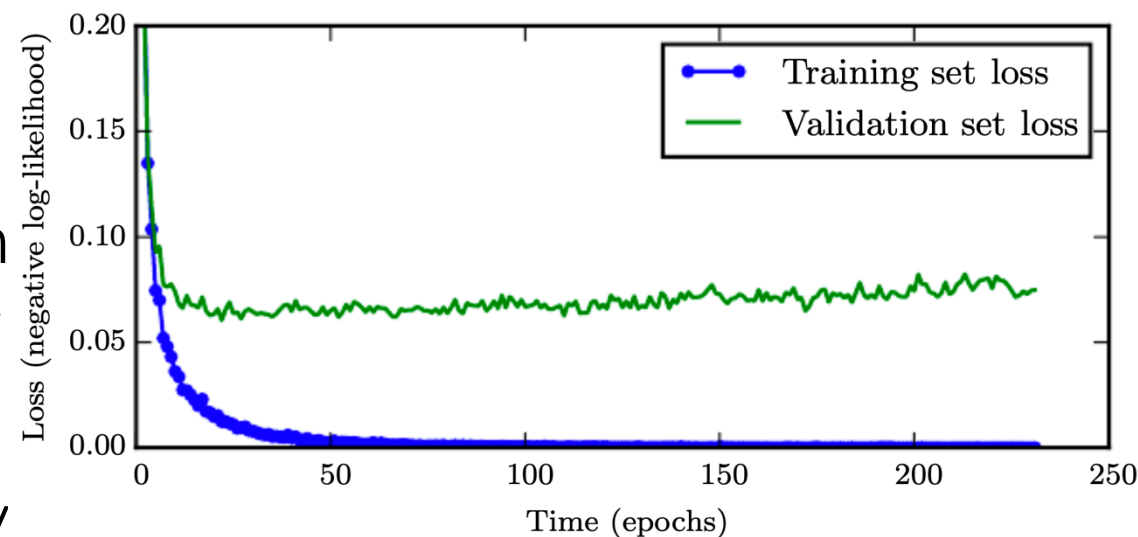
- Unsupervised transfer learning
 - Network is primary trained over a large data set without labels to understand the characteristics of the target domain
 - Afterwards, the network is trained on a specific data set related to the target task and domain
 - It is used when the labelling process is costly

Regularization

- Fine-tuning
 - Transfer learning variation in which a pre-trained backbone is further trained over a small data set for a specific task
 - Generally, only deeper layers of the network are trained and not the initial ones
 - Dynamic techniques to adaptively vary the learning rate, are used
 - It is also used for domain adaptation

Regularization

- Early stopping
 - Training that lasts for too many epochs leads to overfitting and reduces the ability to generalize
 - Early stopping is an algorithm to stop the training when the loss value over a suitable validation set, starts to increase, forming an elbow method



Regularization

- Early stopping

- A patience value is used for this purpose
- It indicates the number of epochs after which, if a decreasing in validation loss is not observed, the training procedure ends

Let n be the number of steps between evaluations.

Let p be the “patience,” the number of times to observe worsening validation set error before giving up.

Let θ_o be the initial parameters.

$\theta \leftarrow \theta_o$

$i \leftarrow 0$

$j \leftarrow 0$

$v \leftarrow \infty$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

while $j < p$ **do**

 Update θ by running the training algorithm for n steps.

$i \leftarrow i + n$

$v' \leftarrow \text{ValidationSetError}(\theta)$

if $v' < v$ **then**

$j \leftarrow 0$

$\theta^* \leftarrow \theta$

$i^* \leftarrow i$

$v \leftarrow v'$

else

$j \leftarrow j + 1$

end if

end while

Best parameters are θ^* , best number of training steps is i^* .

Regularization

- Early stopping

- Train the model over a training set

#1: training per i^* epochs

#2: training from pre-trained weights θ^*

#3: early stopping inserted in the training pipeline, the algorithm is automatically stopped

- The obtained result is analogous to the regularization effect given by the usage of L^2 norm

Regularization

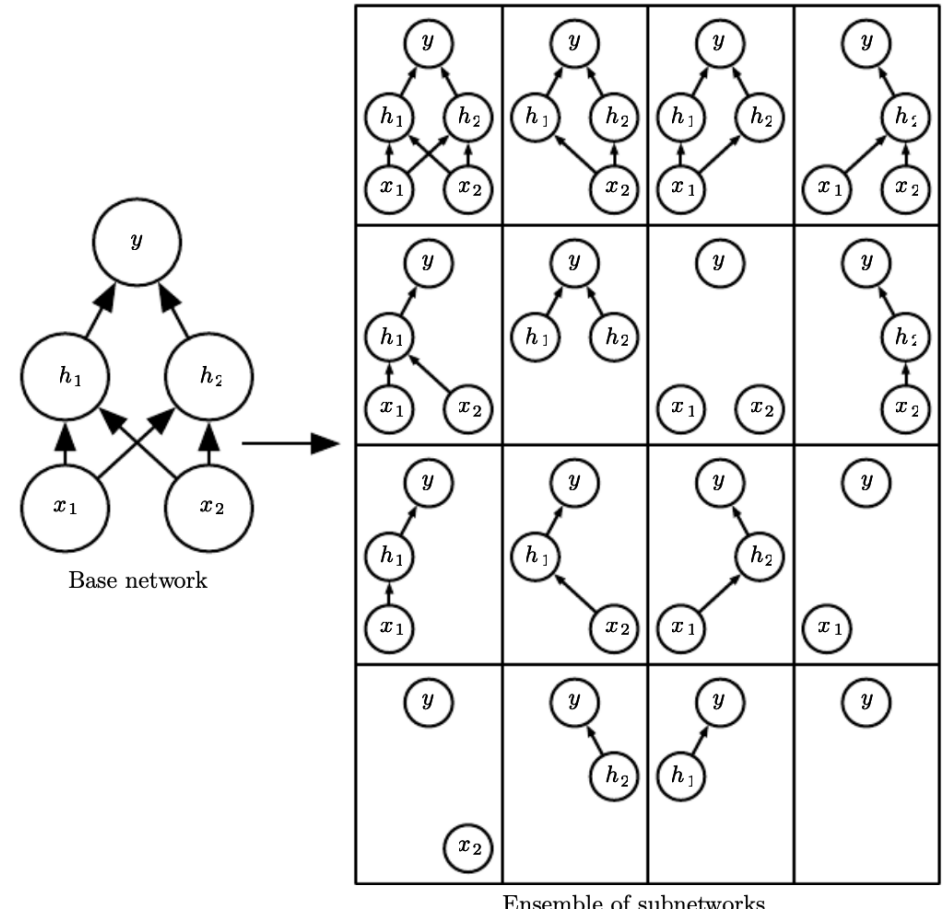
- Dropout

- Statistic technique in which links between units are pruned (drop)
- It effectively generates an ensemble of models that are trained simultaneously
- Increased accuracy on the test set using the mean or the majority of the ensemble's predictions

Regularization

- Dropout

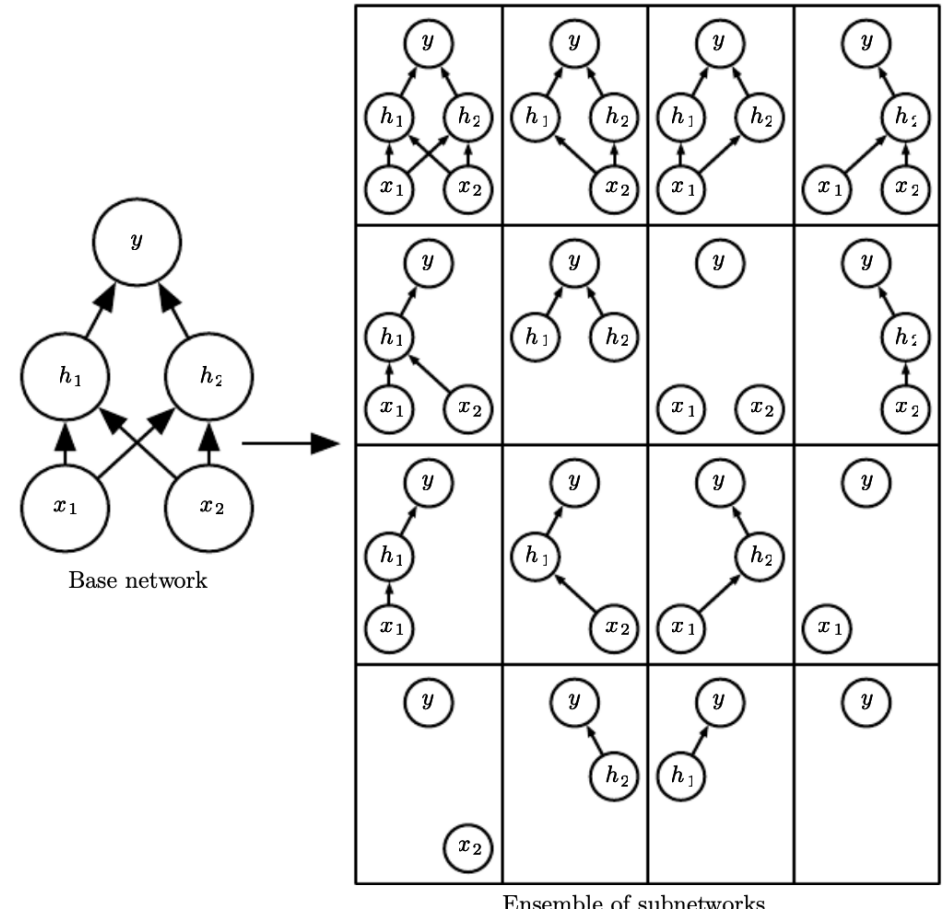
- By gradually removing the different connections, the sub-networks of the starting network are obtained
- Each unit will be included in the network with some probability
 - This probability is an hyperparameter
 - Dropout 0.8 for input units
 - Dropout 0.5 for hidden drives



Regularization

- Dropout

- μ is the binary vector which mask the units' activations
- Each unit will be activated as $\mu_i h_i$
- For each minibatch, a different mask vector is random sampled with a probability $p(\mu)$
- Prediction of each model $p(y | x, \mu)$ is considered in the ensemble method



Optimization

- Optimization in DNN involves all the algorithms and techniques to accelerate the convergence of the minimization procedure over the loss function
- Theoretically, the objective is to minimize the empirical risk, which is the expected value of the loss, assuming the training samples are picked from an empirical distribution and not from the real one

$$\mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}(\mathbf{x}, y)} [L(f(\mathbf{x}; \boldsymbol{\theta}), y)] = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), y^{(i)})$$

Optimization

- Minimization approaches assume that the objective function is locally approximable with a quadratic function (gradient descend)
- But the direct minimization of empirical risk typically leads to overfitting
 - Calculating *all* gradients is very expensive
 - The network will train well for high-capacity parameter configurations and will tend to minimize training error alone
- The focus is towards the minimization of a different function, by selecting a minibatch of different samples over which Stochastic Gradient Descent (SGD) or its variants are applied

Optimization

- SGD

Require: Learning rate schedule $\epsilon_1, \epsilon_2, \dots$

Require: Initial parameter θ

$k \leftarrow 1$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

Apply update: $\theta \leftarrow \theta - \epsilon_k \hat{\mathbf{g}}$

$k \leftarrow k + 1$

end while

Optimization

- SGD
 - $\hat{\mathbf{g}}$ is calculated over a minibatch and allows to the algorithm a good convergence with low computational cost
 - Learning rate ε is usually linearly decreased
 - Minibatch serves as regularization
 - It is suitable to be parallelly executed over a multi-core architecture
 - Usually are used power of two values, from 32 to 256

Optimization

- SGD with momentum

- A momentum term is used to accumulate a moving average of the past gradients

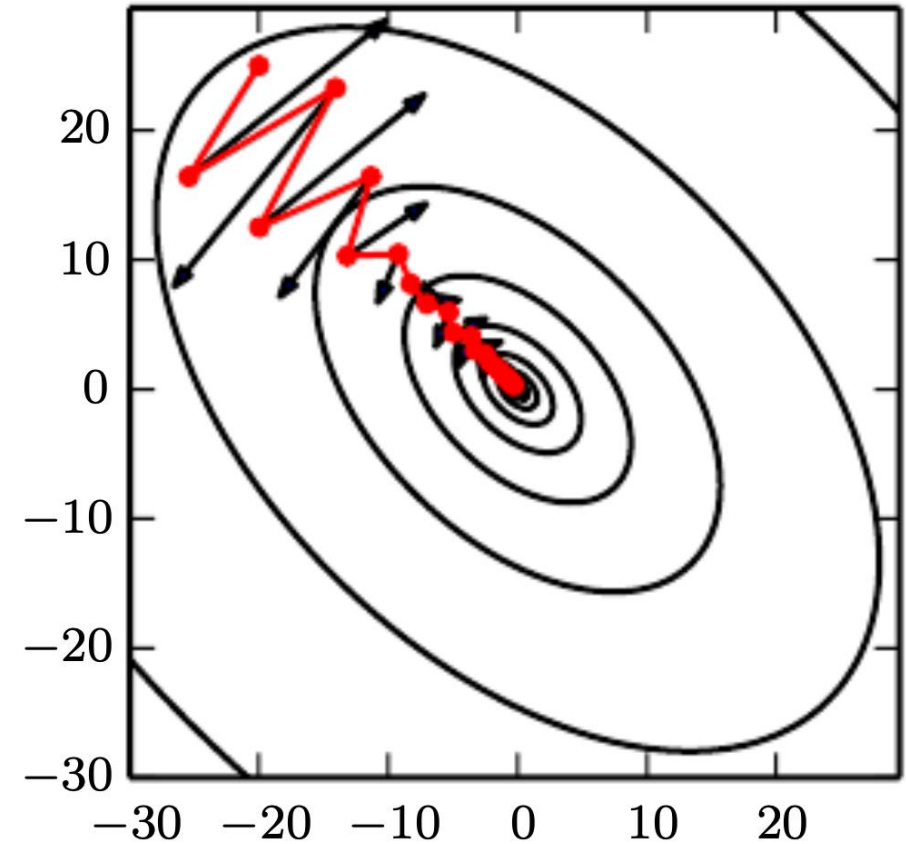
$$\begin{aligned} \mathbf{v} &\leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^m L(\mathbf{f}(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{y}^{(i)}) \right) \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{v}. \end{aligned}$$

- Nesterov momentum also involves the updates of $\boldsymbol{\theta}$

$$\begin{aligned} \mathbf{v} &\leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left[\frac{1}{m} \sum_{i=1}^m L(\mathbf{f}(\mathbf{x}^{(i)}; \boldsymbol{\theta} + \alpha \mathbf{v}), \mathbf{y}^{(i)}) \right] \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{v}, \end{aligned}$$

Optimization

- SGD with momentum
 - Momentum makes the solution tend in directions with less curvature to prevent the solution from diverging

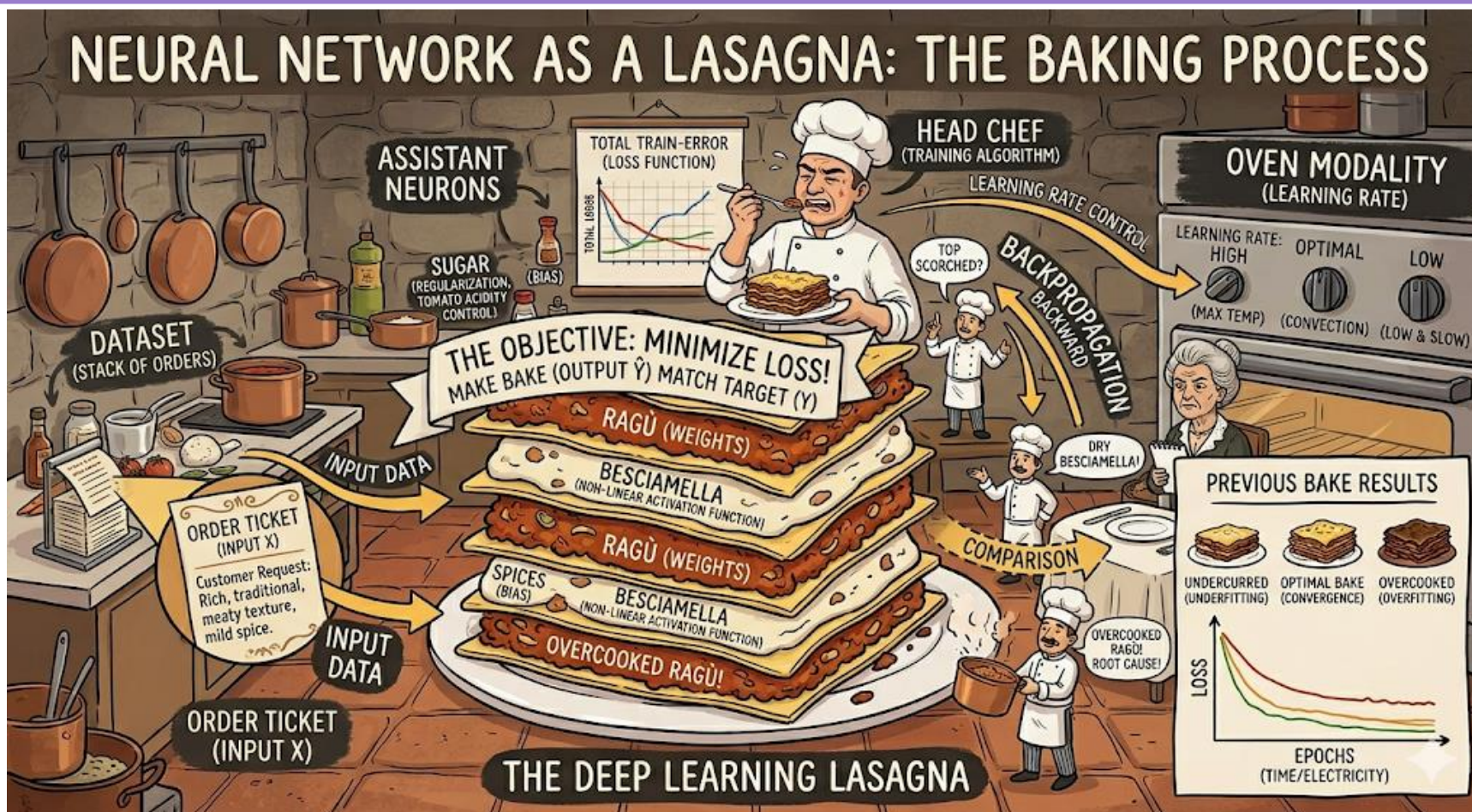


Black → gradient descend
Red → momentum

Optimization

- Algorithms with variable learning rate
 - Learning rate is adapted via scaling with a term linked with the accumulation of the squares of the gradients along training (momentum of the second order of $\mathbf{g}$)
 - AdaGrad uses the entire history of the gradients
 - RMSProp uses a mobile average over a window with a decay term and the Nesterov momentum
 - Adam uses a regularization with momentum of first and second order of $\mathbf{g}$

The Lasagna Neural Network



Principal Neural Network

- On the basis on the simple unit and principles, more complex networks can be built and implemented for target applications and data
 - Convolutional Neural Networks
 - Autoencoder
 - Generative Adversarial Networks
 - Recurrent Neural Networks
 - Long Short-Term Memory
 - Transformers

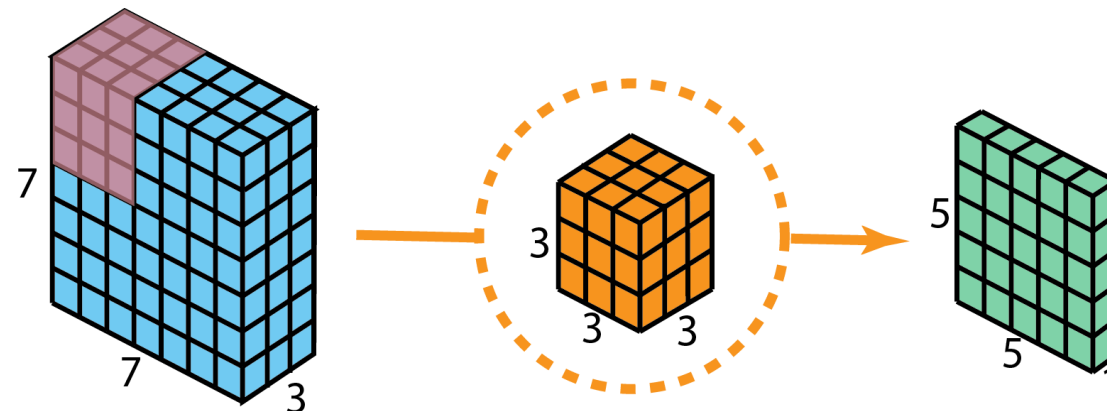
Convolutional Neural Networks

- Convolutional Neural Networks (CNN) are a particular type of neural networks which aims to analyze data with a grid-like structure
 - Images
 - Monodimensional sequences with a fixed analysis
 - Sampling in temporal series
 - Discrete series (sentences)
 - Molecules
 - ...

Convolutional Neural Networks

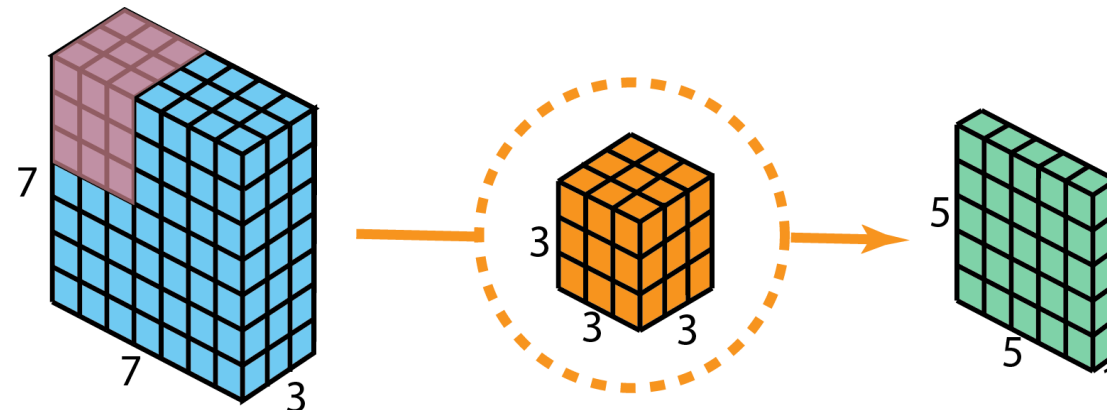
- The convolutional operator is used between two subsequential layers

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$



Convolutional Neural Networks

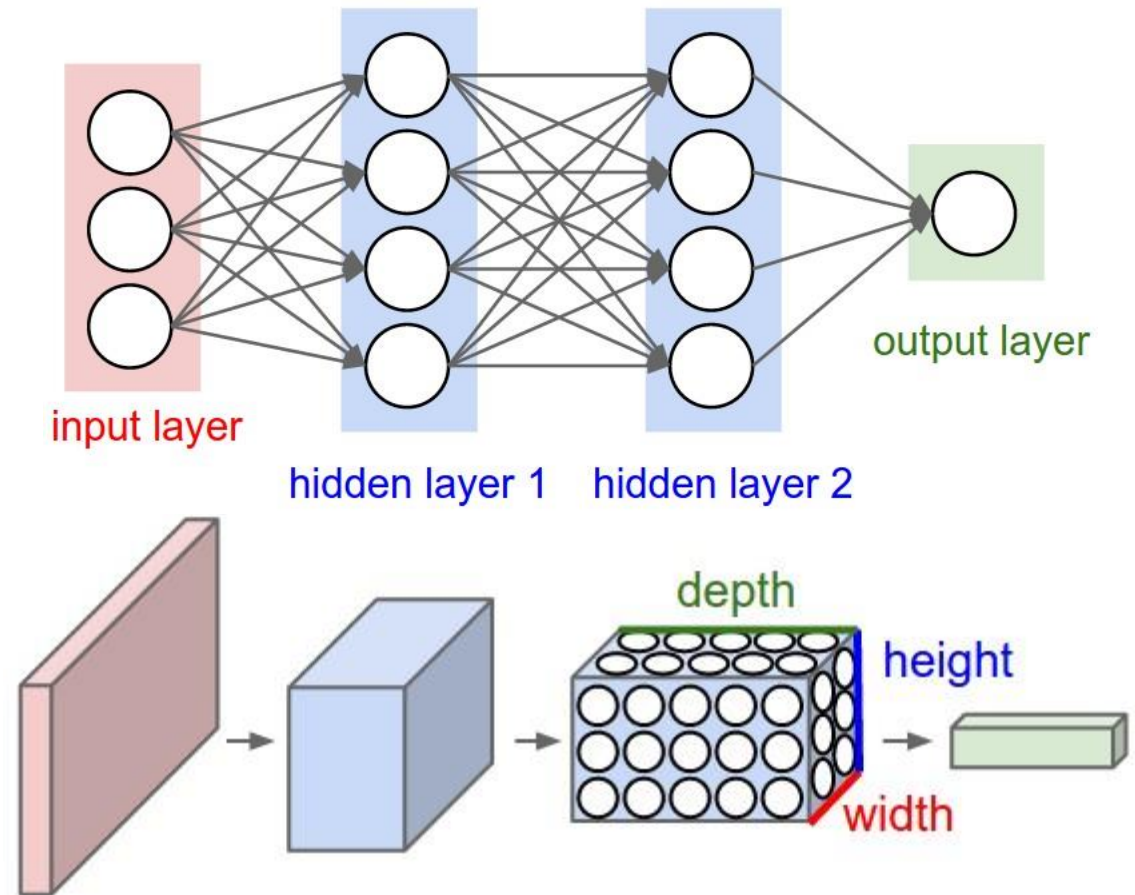
- The kernel is always applied to an n -dimensional tensor, also in the case it is monodimensional
- Usually, dimensions of the input tensor $\mathbf{X}_{b,w,h,d}$ are batch, width, height and depth (or channels), order may vary
 - Monodimensional vector $\mathbf{x} \in \mathbb{R}^d \rightarrow \mathbf{X}_{b,d,1,1}$



Convolutional Neural Networks

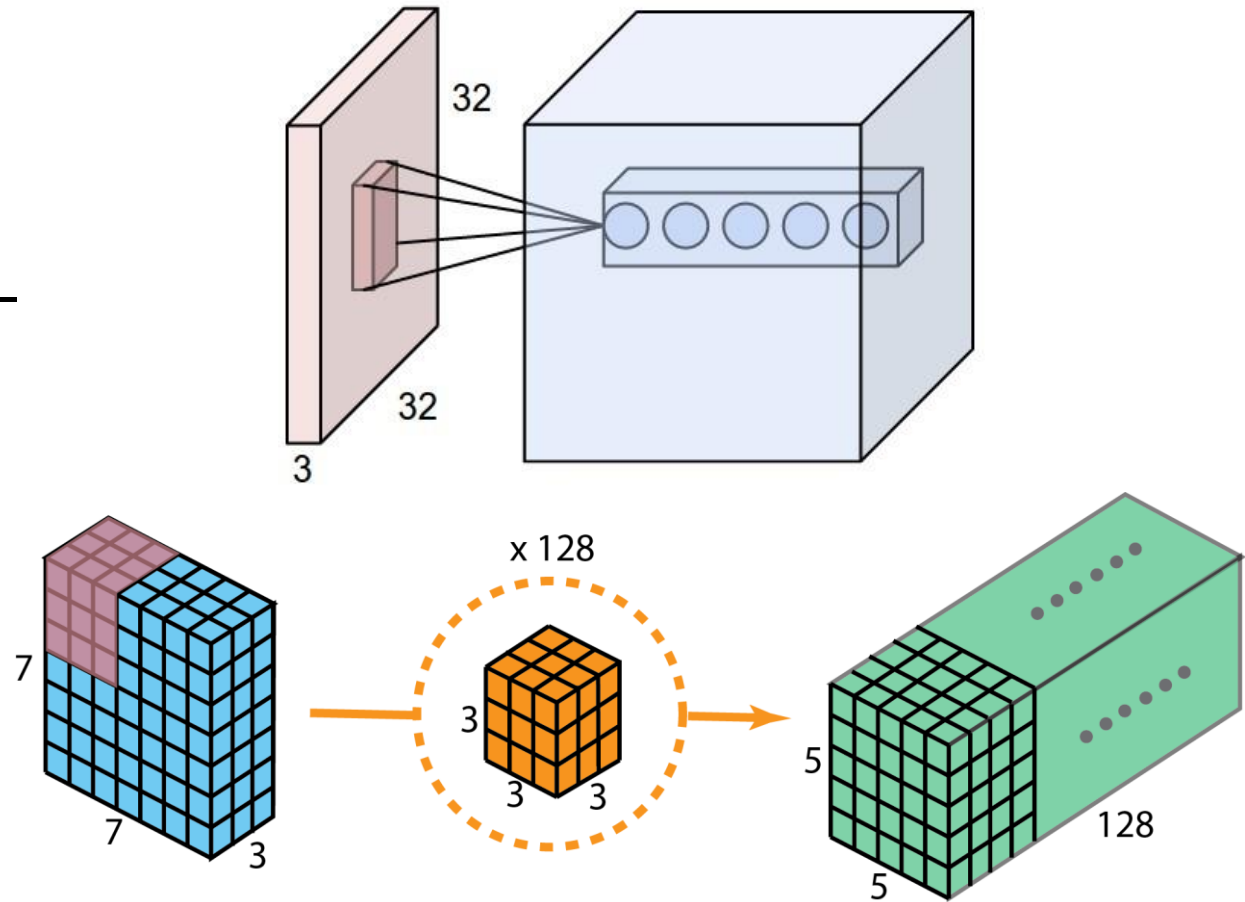
- Tensor's structure explicitly derives from images' dimensionality where depth is associated with R, G, B channels
- Batch dimension is not directly involved in the convolutional operation

$$Y_{i,j,k} = \sum_l \sum_m \sum_n X_{i-l,j-m,k-n} K_{l,m,n}$$



Convolutional Neural Networks

- The dimension of the filter, s , along the grid dimension, is an hyperparameter
- Along the depth direction, the convolutional operation is fully-connected and equal to the depth d of the input
- Each convolutional layer can have a custom depth, which can be different from the one of the other layers, and it is an additional hyperparameter



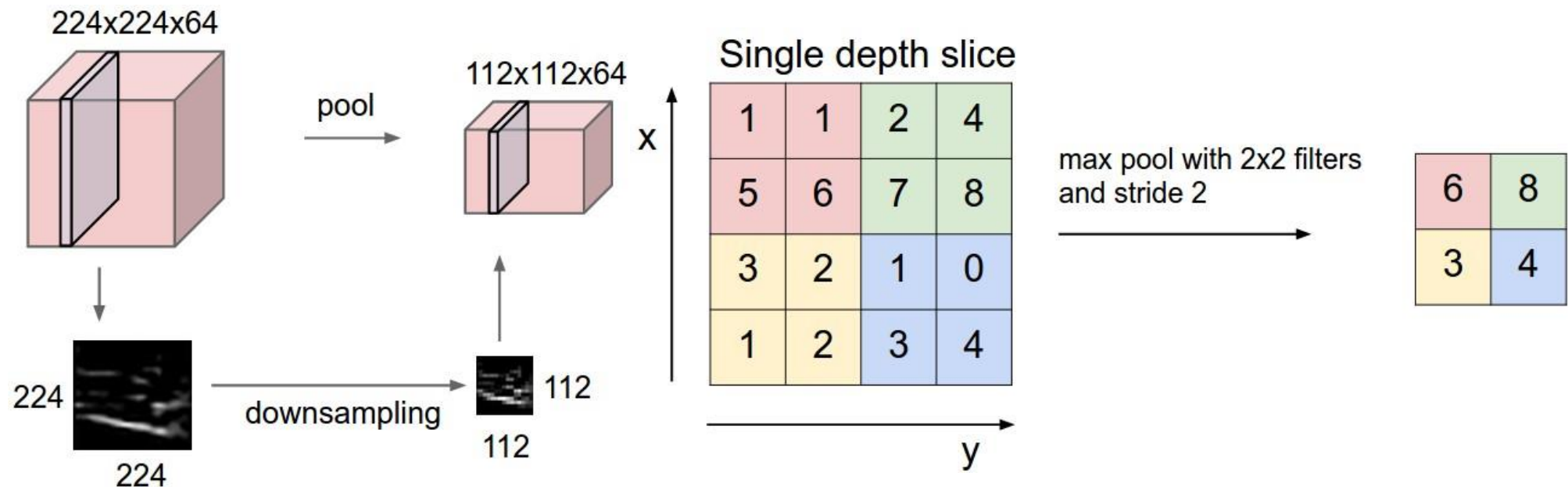
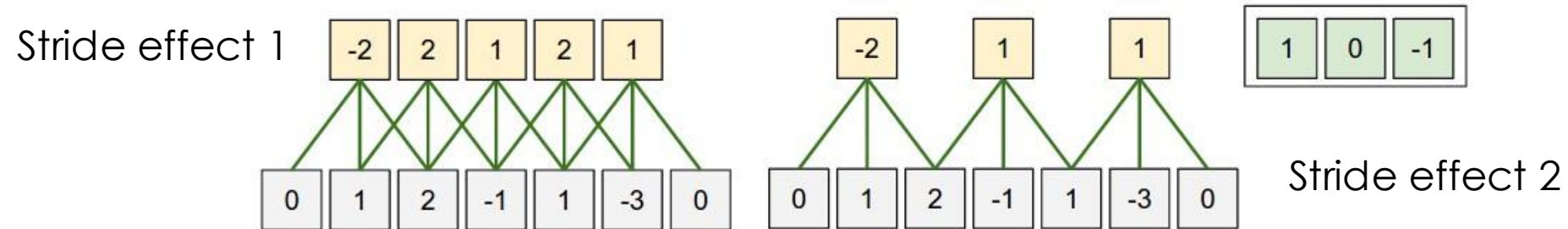
Convolutional Neural Networks

- A *complex* convolutional layer is a sequence of three sub-layers
 - Convolutional: receives the inputs over a limited region and serves to extract local features
 - Diverse feature maps are extracted along depth dimension
 - ReLU: classical DNN layer non non linearity
 - Usually the complex layer is just Conv + ReLU
 - Pooling (optional): performs a sub-sampling over inputs to obtain a scaled and reduced feature map
 - The objective is to obtain aggregated features, position-invariant

Convolutional Neural Networks

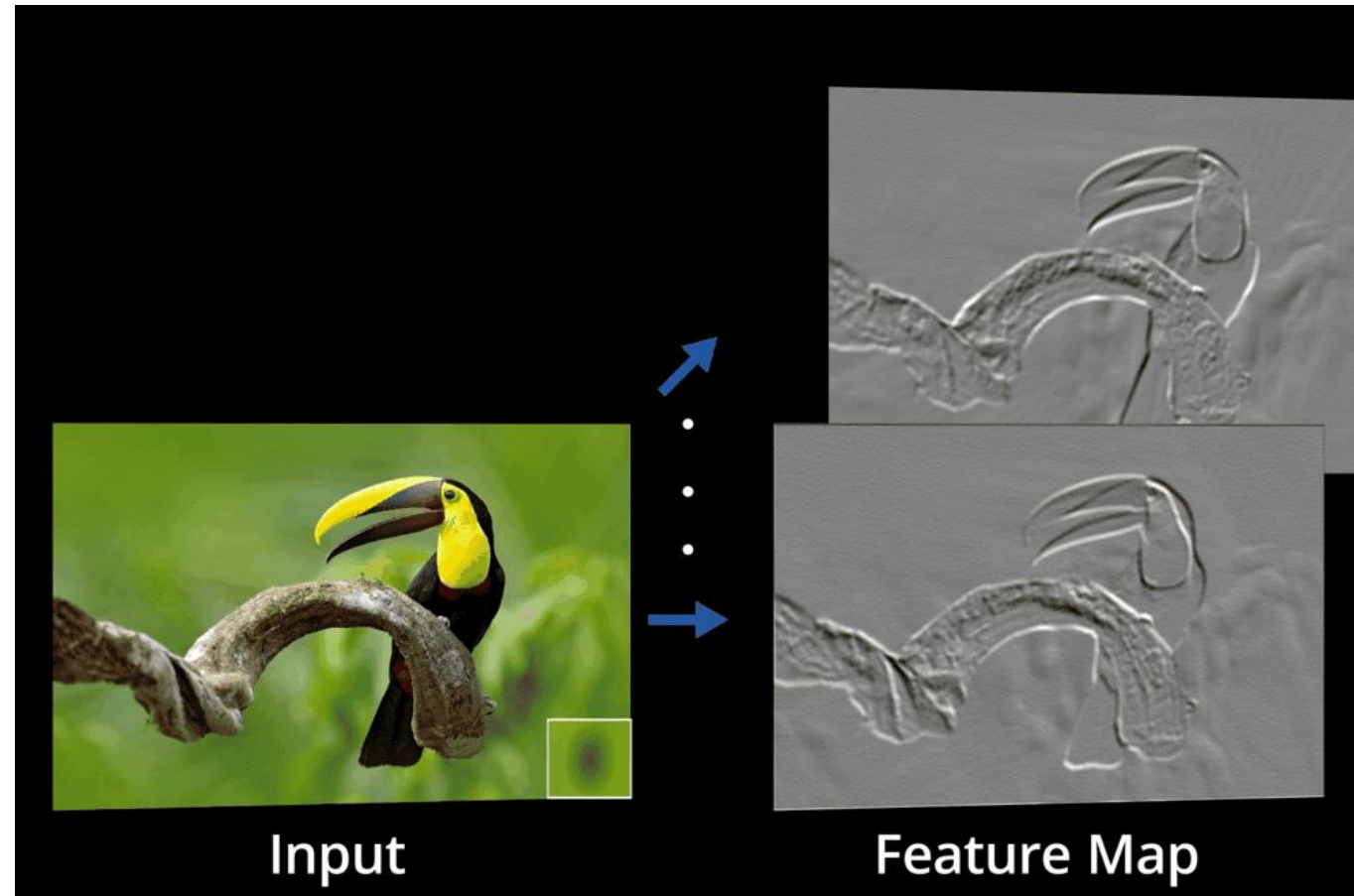
- Pooling layer filters the feature map in a small neighbour (e.g. 2x2) via filters application.
 - Max pooling
 - Average pooling
- Sub-sampling can be obtained via a stride application, which is a skip step in applying convolutional kernels or pooling
 - A zero-padding sometime is needed, which is a null-values frame around the feature map, thus the kernel step and its dimension are compliant with the feature map one

Convolutional Neural Networks



Convolutional Neural Networks

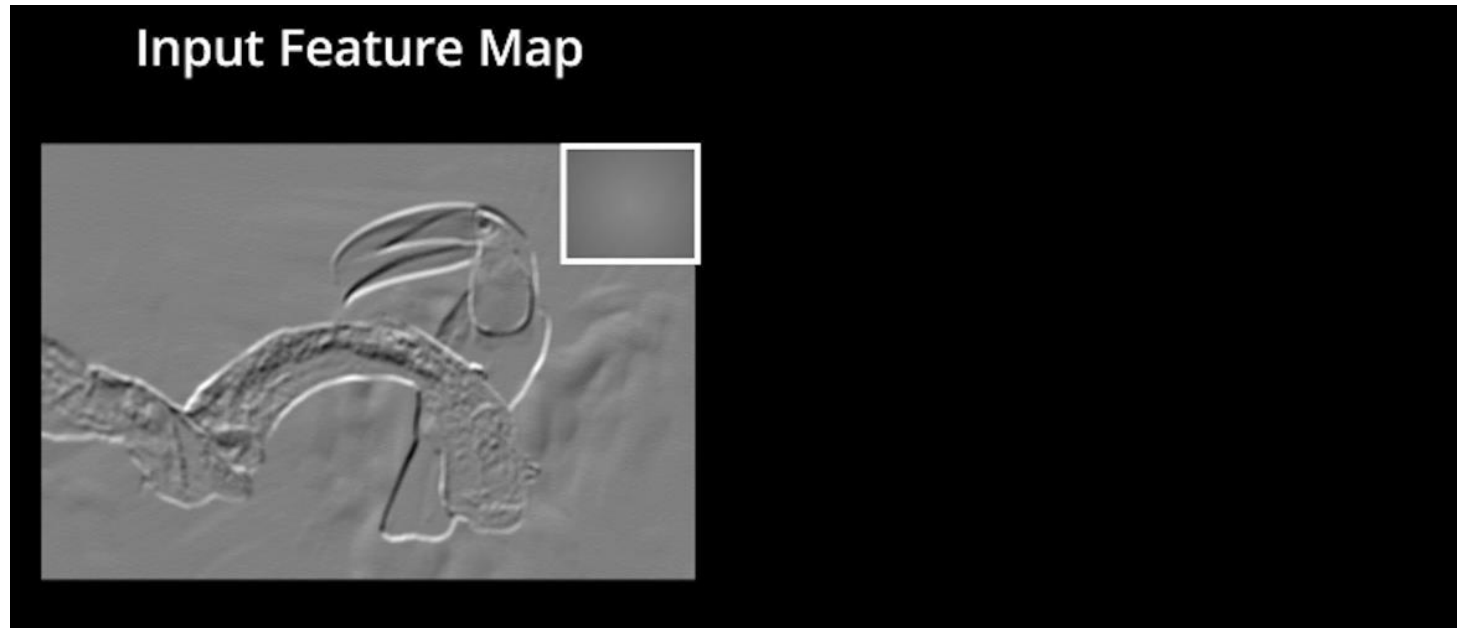
Convolutional
layer



<https://www.simplilearn.com/tutorials/deep-learning-tutorial/convolutional-neural-network>

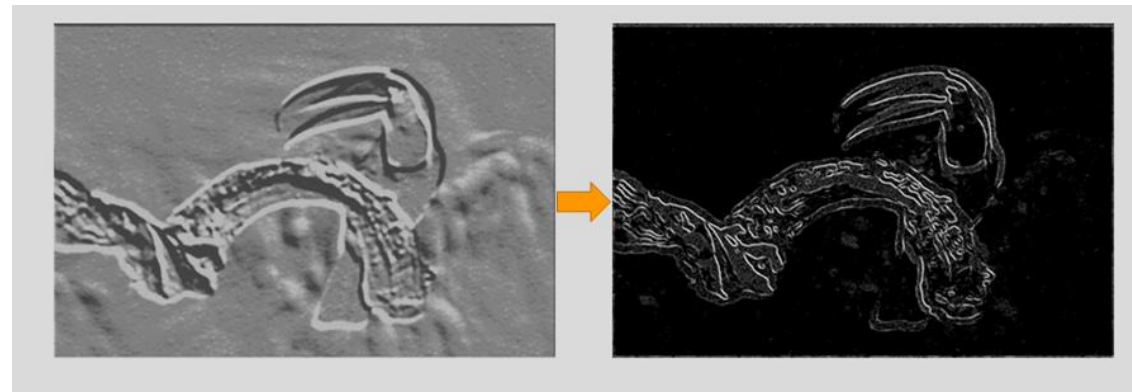
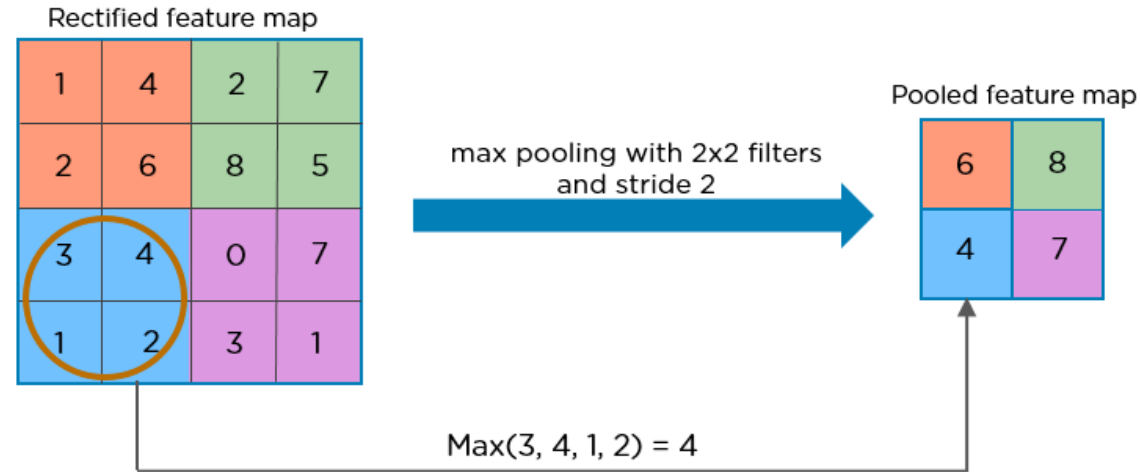
Convolutional Neural Networks

ReLU
layer



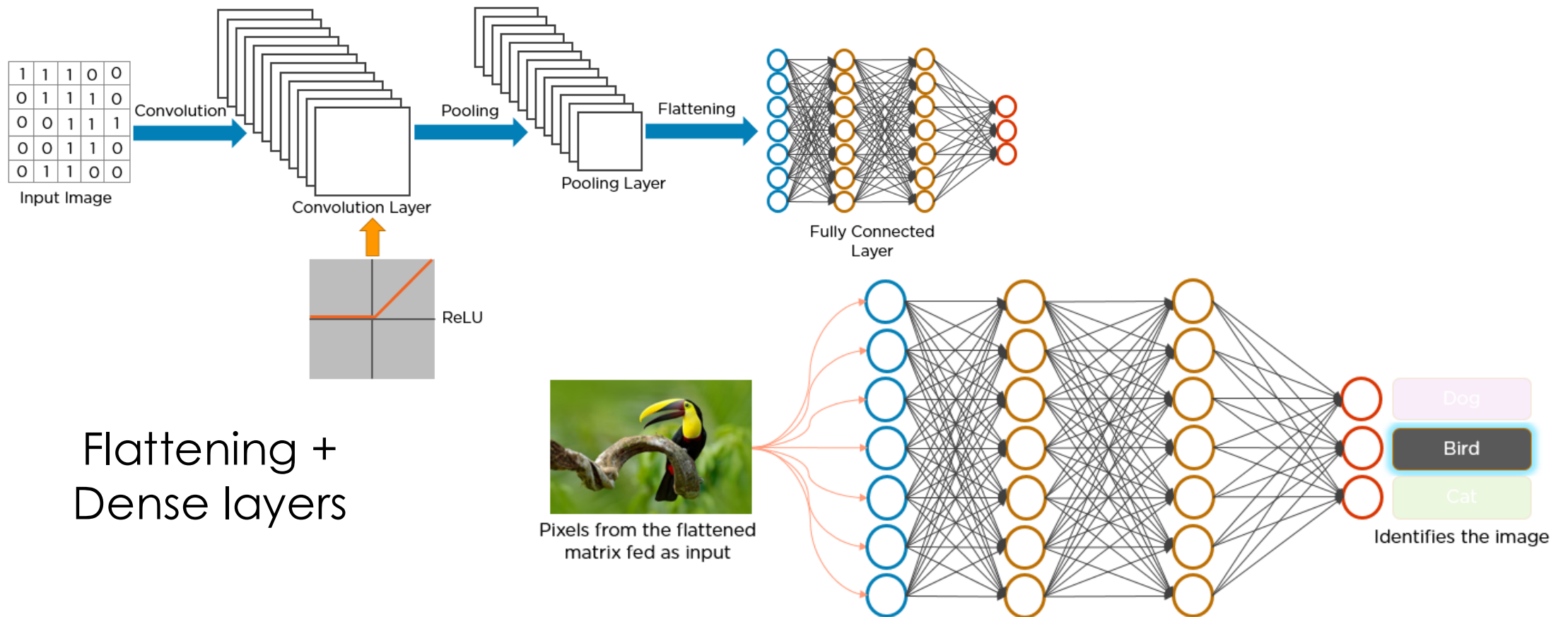
Convolutional Neural Networks

Max Pool
2x2 layer



<https://www.simplilearn.com/tutorials/deep-learning-tutorial/convolutional-neural-network>

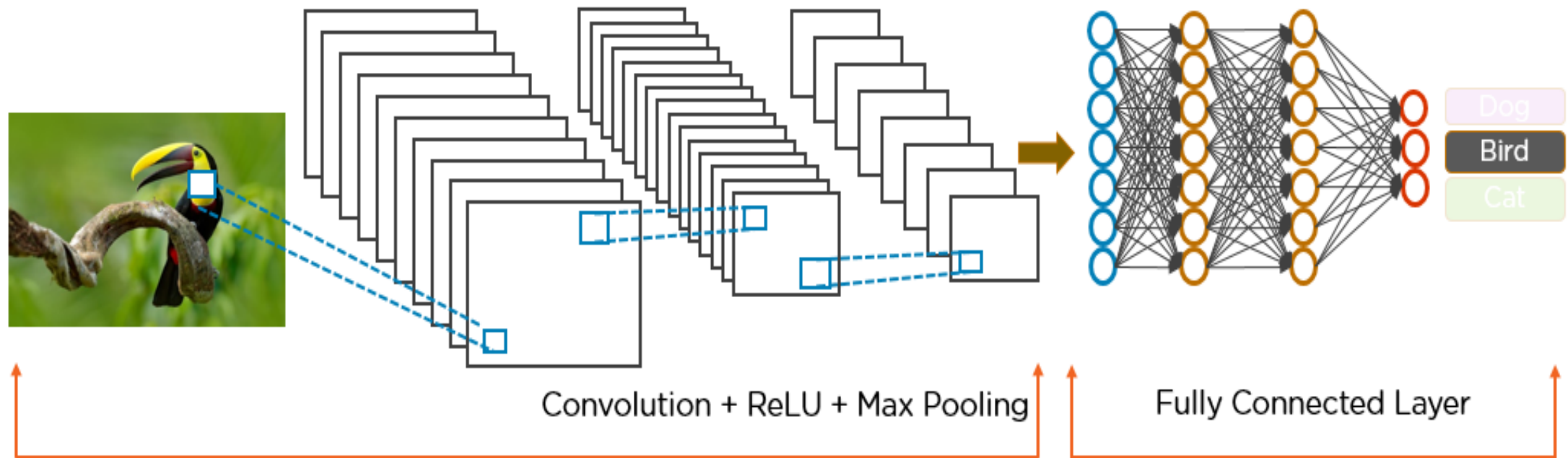
Convolutional Neural Networks



<https://www.simplilearn.com/tutorials/deep-learning-tutorial/convolutional-neural-network>

Convolutional Neural Networks

- The presence of multiple complex layers, typically with many feature maps and with different levels of sub-sampling, allows to extract features that are hierarchically increasingly aggregated and invariant at scale and location



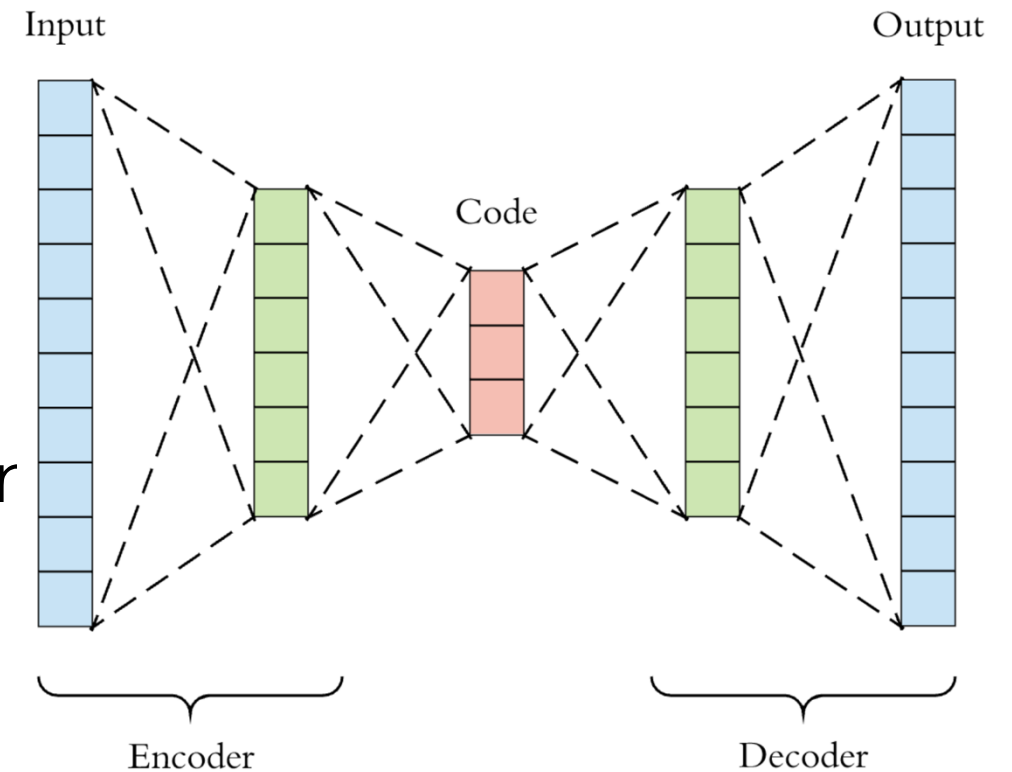
Feature Extraction in multiple hidden layers

Classification in the output layer

<https://www.simplilearn.com/tutorials/deep-learning-tutorial/convolutional-neural-network>

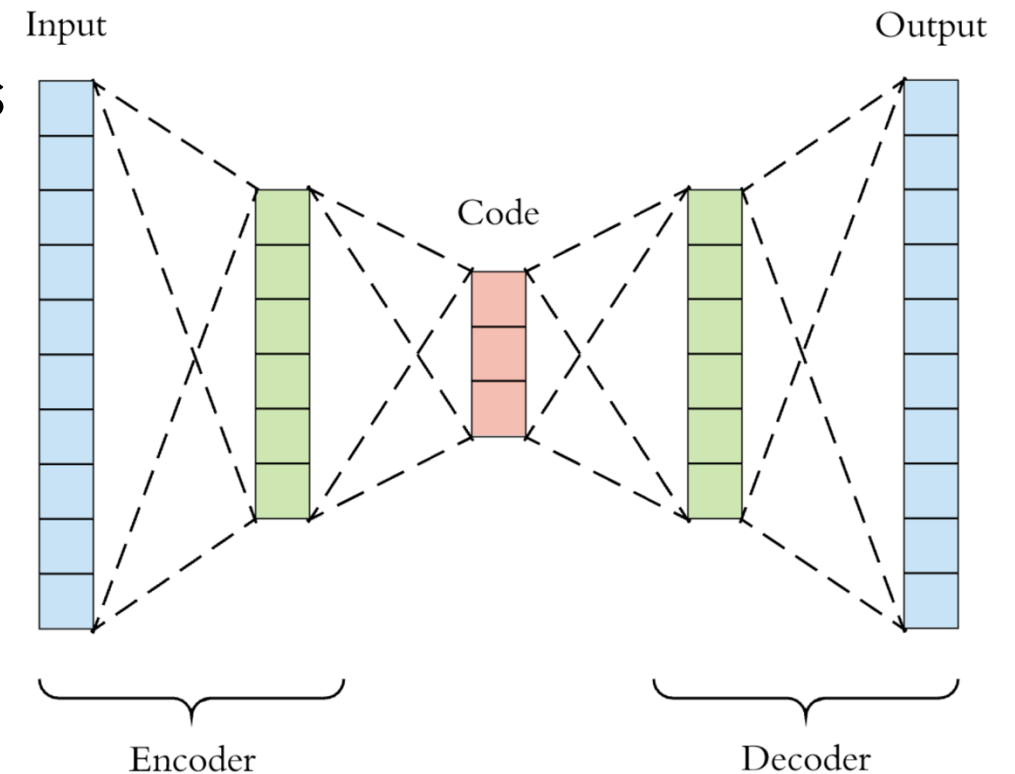
Autoencoder

- An autoencoder is a pair of two CNNs in which two symmetric componets are defined, called encoder and decoder.
- Encoders and decoders are trained over *equal* input/output pairs
- The most internal layer, (code layer) learns a representation of or the inputs in a latent space with a *lower dimensionality*
 - Linear activations → PCA
 - ReLU are used



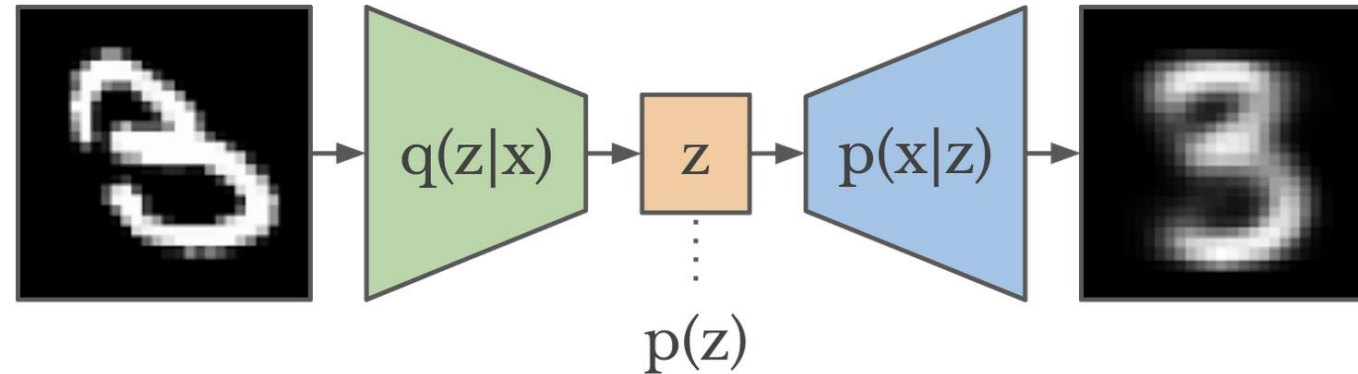
Autoencoder

- Denoising autoencoder
 - Learn upon noisy data, but use a loss referred to the noisy data point
 - Regularization to reduce overfit
- Sparse autoencoder
 - Learn a sparse coding vector with a lots of units, but not all of them are activated
 - Regularization to reduce overfit
- Contractive autoencoder
 - Learning with an explicit regularization over the loss function



Variational Autoencoder

- The Variational autoencoder (VAE) estimates a probability distribution $q(\mathbf{z} | \mathbf{x})$ from which generate the outputs $\mathbf{x}'$ observed from the latent variable $\mathbf{z}$
- VAE is trained to make a MAP estimation of $q(\mathbf{z} | \mathbf{x})$ with a Gaussian prior $p(\mathbf{z})$



Variational Autoencoder

- Data can be considered independent, thus the loss will be the sum of the single contributions for each data points
- Loss function has two components:
 - Cross-entropy of the distribution $p_{\phi}(x_i | z)$ with z drawn from $q_{\theta}(z | x_i)$
 - Kullback-Leibler divergence with measure the dissimilarity of $q_{\theta}(z | x_i)$ from the prior
- From Bayesian inference and theorem, VAE learns joint distribution probability $p(x, z) = p(x | z)p(z)$
- The optimal distribution $q_{\lambda}(z | x)$ minimizes the Kullback-Leibler divergence from $p(z | x)$

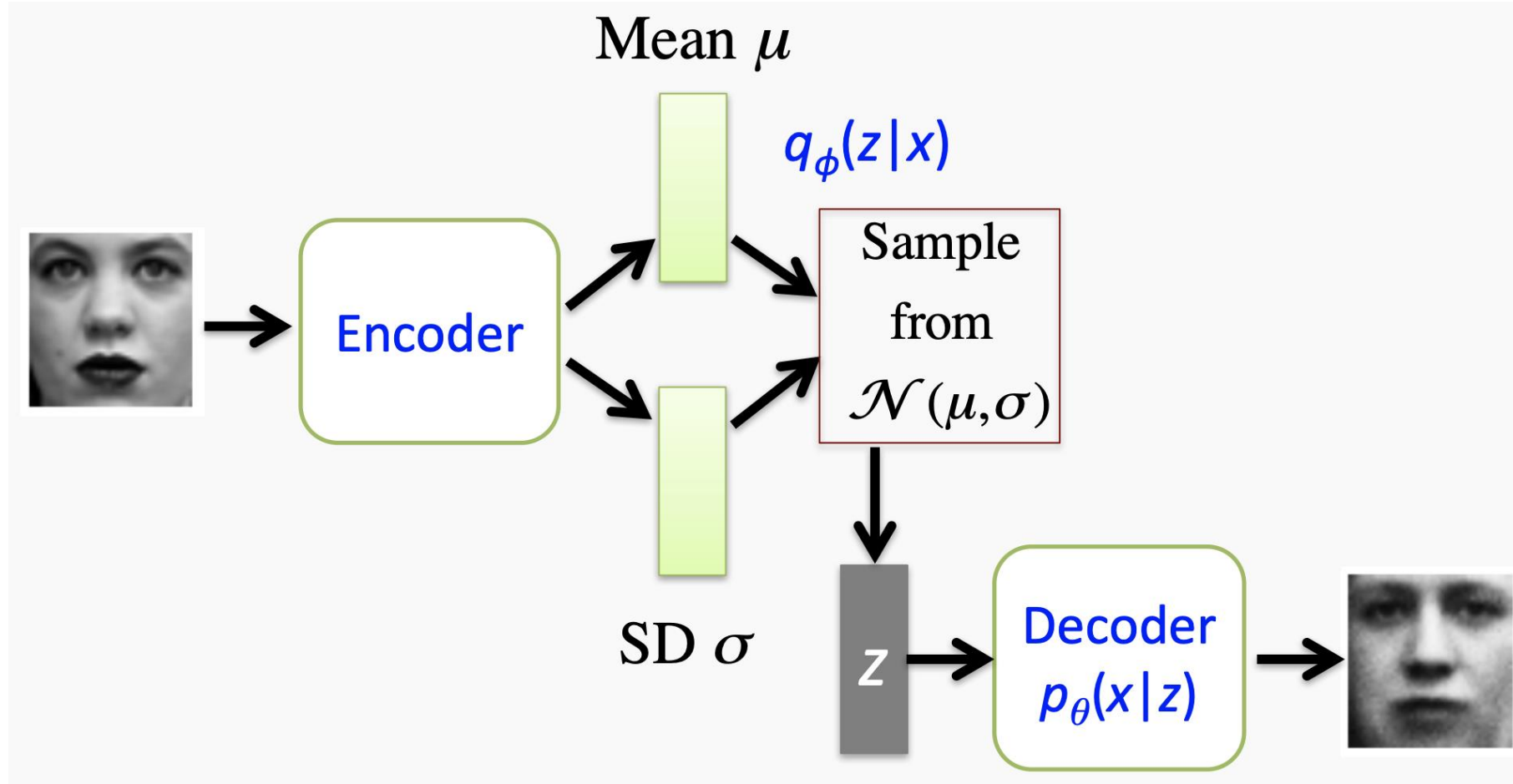
Variational Autoencoder

- Sampling from z is re-parametrized starting from mean and variance vectors and an error over variance extracted from the normal distribution $\mathcal{N}(0,1)$, to efficiently calculate the gradients

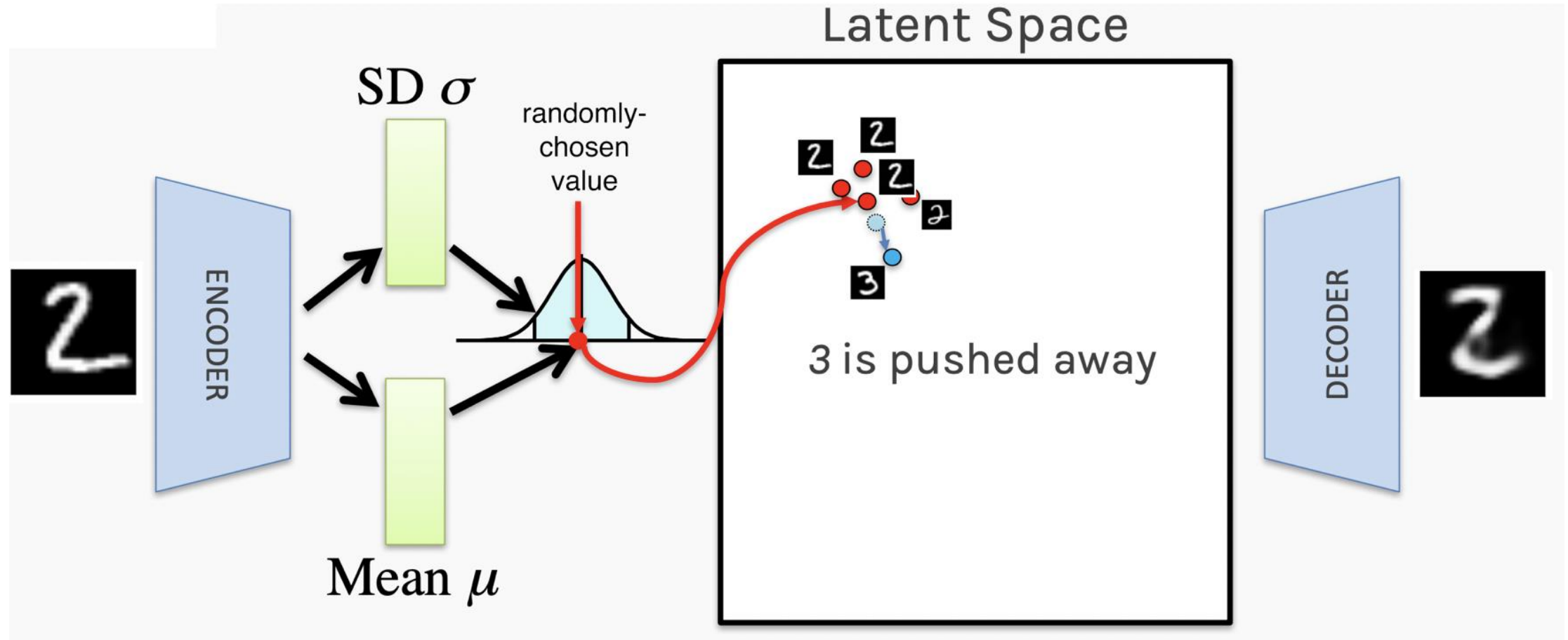
$$z = \mu + \sigma * \epsilon; \quad \epsilon \sim \mathcal{N}(0, 1)$$

- VAE can also be used to generate syntetic and realistic samples from the input data space
 - Celebrity faces generation, deep-fake

Variational Autoencoder



Variational Autoencoder



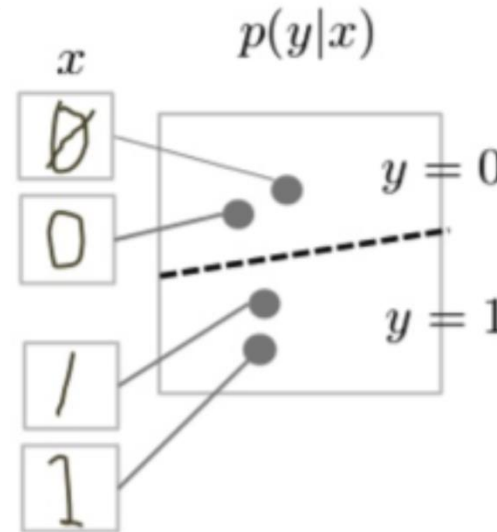
Generative Adversarial Networks

- Generative Adversarial Networks (GAN) are architectures tailored for generating syntetic and realistic samples
- GAN uses two networks, a generator and a discriminator, to compete against each other in order to generate realistic samples

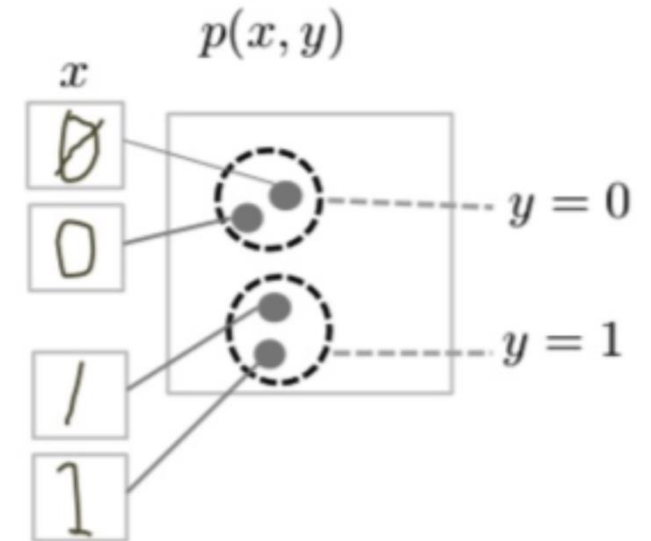
Generative Adversarial Networks

- Generator is not supervised and learns the joint probability $p(\mathbf{x}, y)$ of inputs and outputs
 - It is trained from a Gaussian noise, to generate realistic samples
- Discriminator is supervised and learns $p(y | \mathbf{x})$
 - It is trained to discriminate real data from synthetic ones produced by the generative model

• Discriminative Model

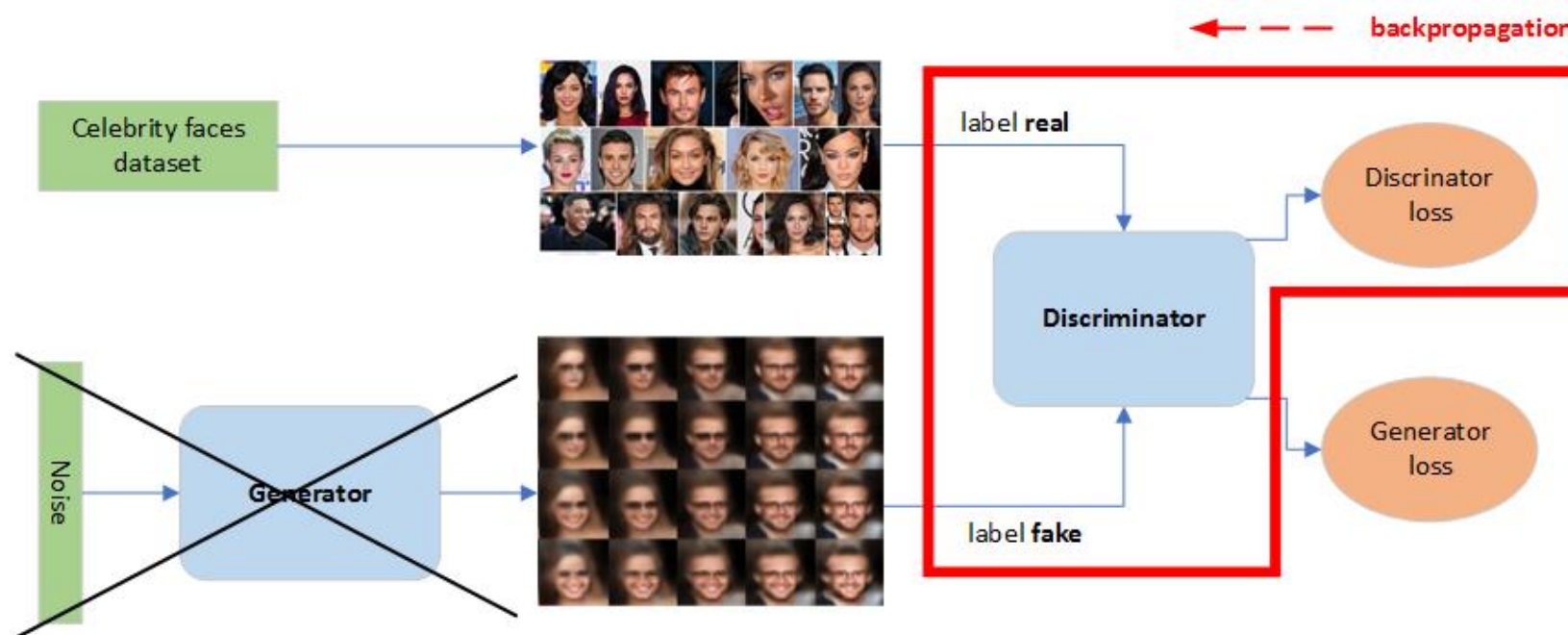


• Generative Model



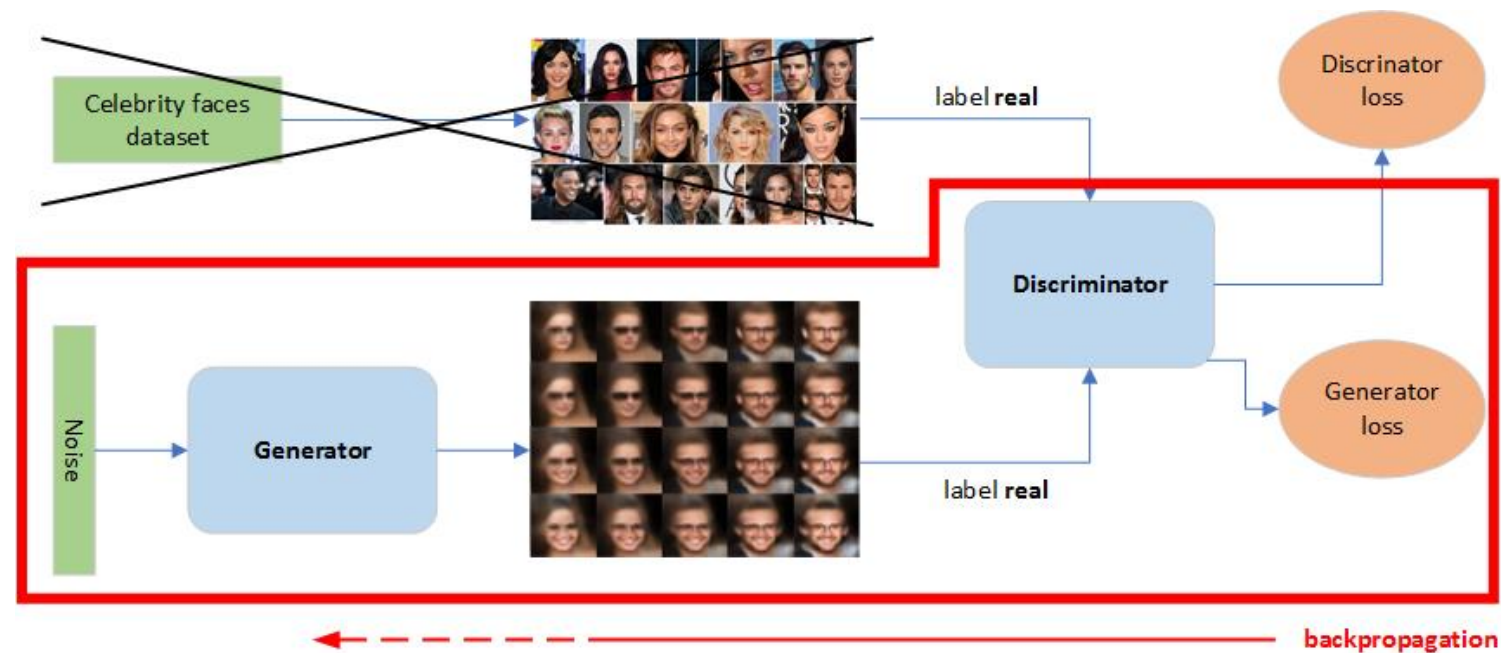
Generative Adversarial Networks

1. Sample m noise data points $\mathbf{z}$ and real data points $\mathbf{x}$
2. Train D over $\mathbf{x}$ and $G(\mathbf{z})$ is not under training



Generative Adversarial Networks

3. Sample again m noise samples $\mathbf{z}$
4. Train G leveraging the loss of $D(G(\mathbf{z}))$

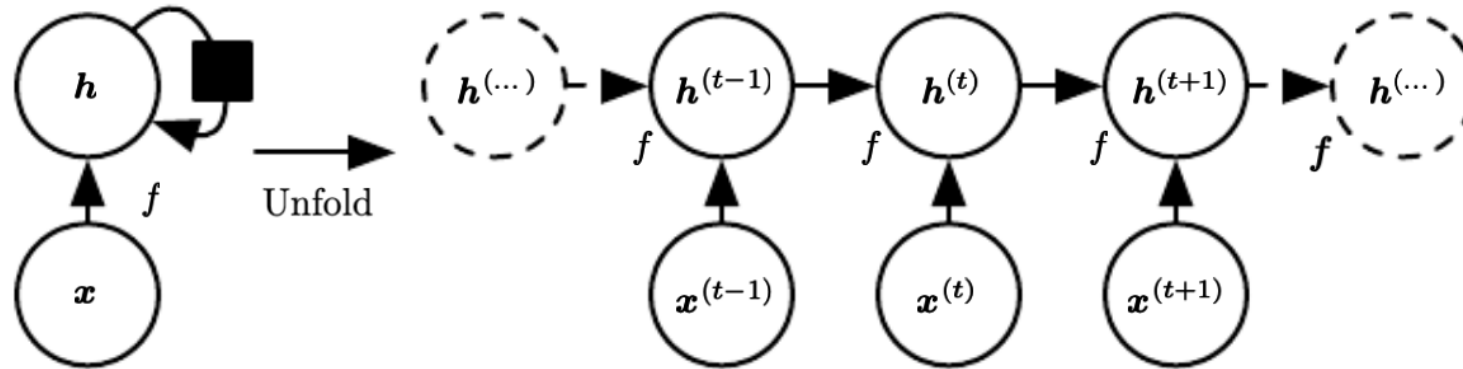


Recurrent Neural Networks

- A Recurrent Neural Network (RNN) is a NN specialized for processing a sequence of values $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(\tau)}$
- Each member of the output is a function of the previous members of the output.
- Each member of the output is produced using the same update rule applied to the previous outputs.

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \boldsymbol{\theta})$$

Recurrent Neural Networks

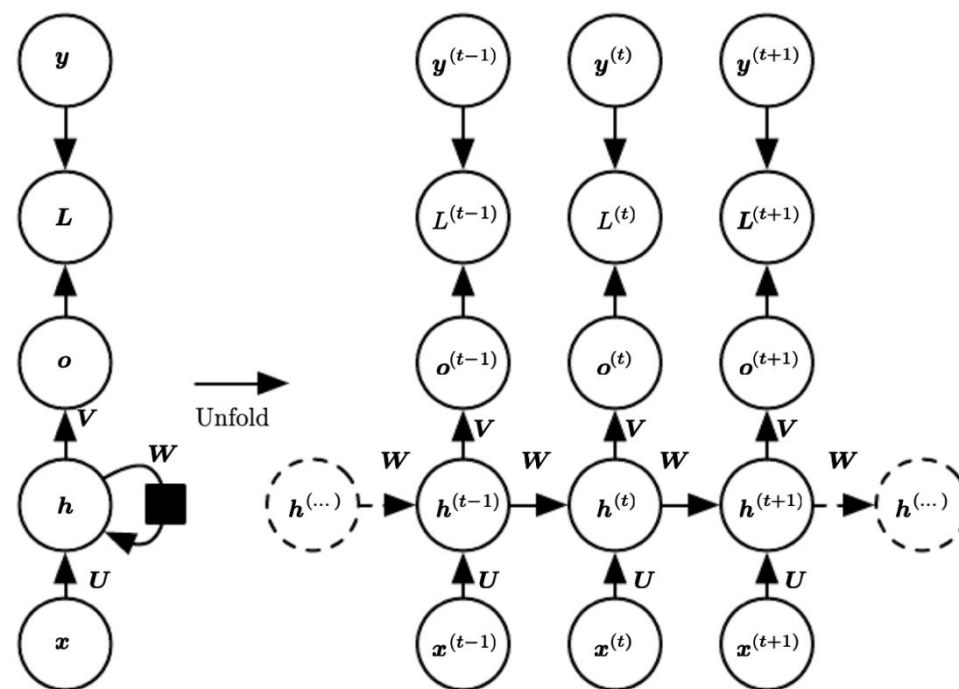


$$\begin{aligned} h^{(t)} &= f(h^{(t-1)}, x^{(t)}; \theta) \\ h^{(t-1)} &= f(h^{(t-2)}, x^{(t-1)}; \theta) \\ &\vdots \\ h^{(1)} &= f(h^{(0)}, x^{(1)}; \theta) \end{aligned}$$

Recurrent Neural Networks

RNNs that produce an output at each time step and have recurrent connections between hidden units.

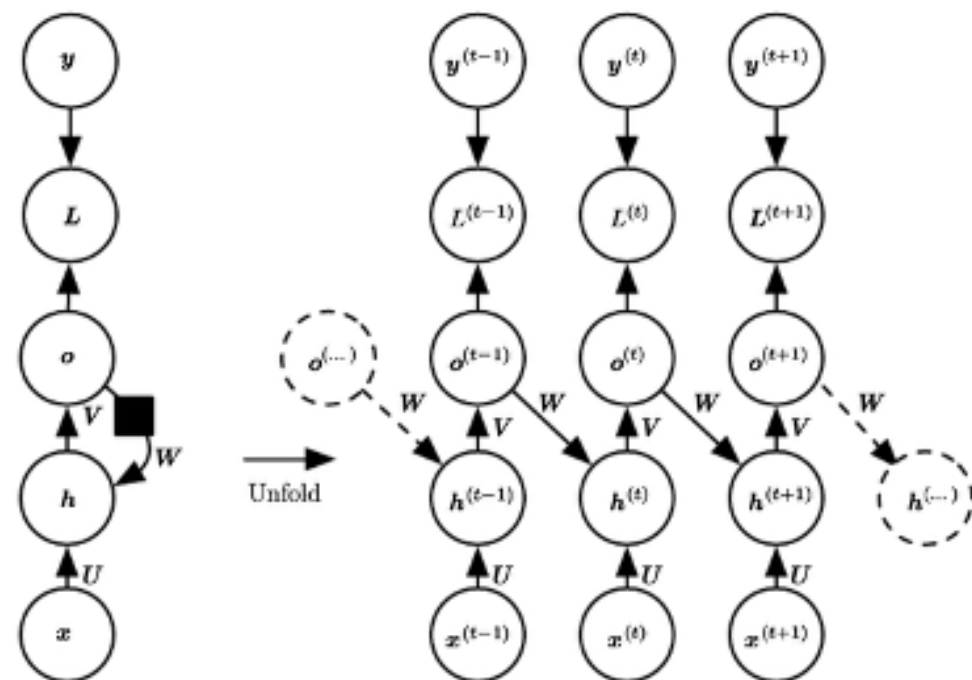
$\mathbf{x} \rightarrow$ input sequence
 $\mathbf{o} \rightarrow$ output values
 $L \rightarrow$ loss
 $\mathbf{h} \rightarrow$ hidden unit
 $\mathbf{y} \rightarrow$ ground truth value



Recurrent Neural Networks

RNNs that produce an output at each time step and have recurrent connections only from the output at one time step to the hidden units at the next time step.

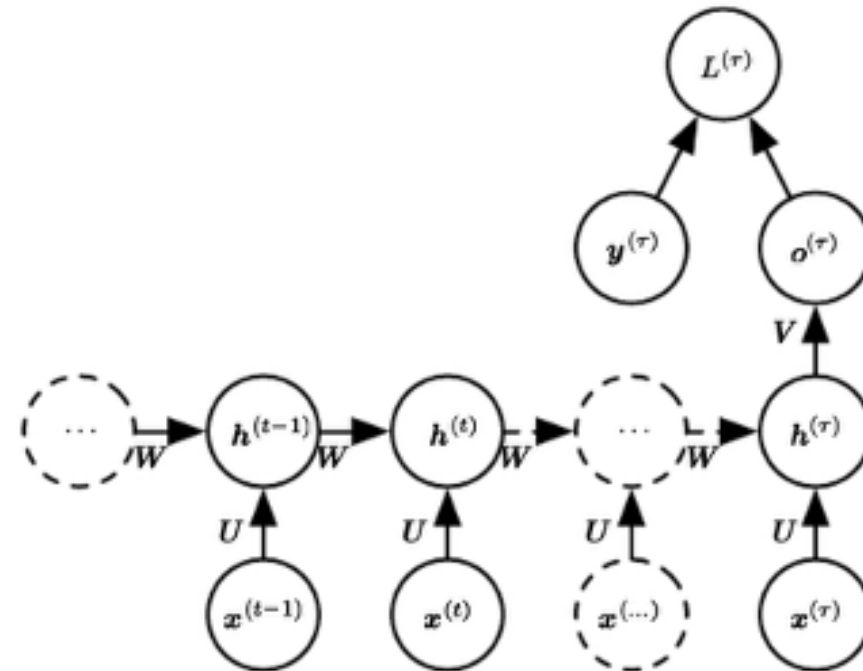
$\mathbf{x} \rightarrow$ input sequence
 $\mathbf{o} \rightarrow$ output values
 $L \rightarrow$ loss
 $\mathbf{h} \rightarrow$ hidden unit
 $\mathbf{y} \rightarrow$ ground truth value



Recurrent Neural Networks

RNNs with recurrent connections between hidden units, that read an entire sequence and then produce a single output.

$\mathbf{x} \rightarrow$ input sequence
 $\mathbf{o} \rightarrow$ output values
 $L \rightarrow$ loss
 $\mathbf{h} \rightarrow$ hidden unit
 $\mathbf{y} \rightarrow$ ground truth value

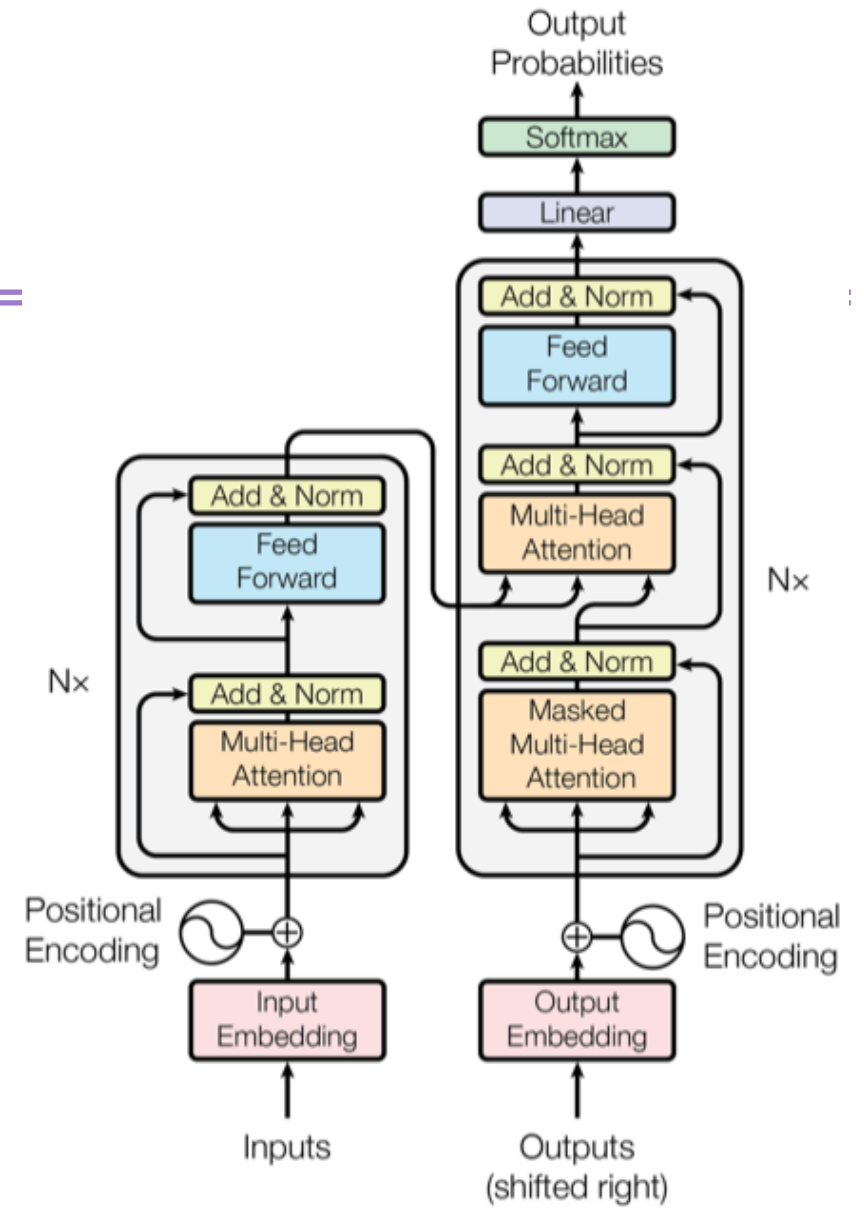


Transformers

Transformers are an architecture which deleted recurrence and relies entirely on an attention mechanism to draw global dependencies between input and output.

$$(\mathbf{z}_1, \dots, \mathbf{z}_n) = \text{encoder}(\mathbf{x}_1, \dots, \mathbf{x}_n)$$

$$(\mathbf{y}_1, \dots, \mathbf{y}_m) = \text{decoder}(\mathbf{z}_1, \dots, \mathbf{z}_n)$$



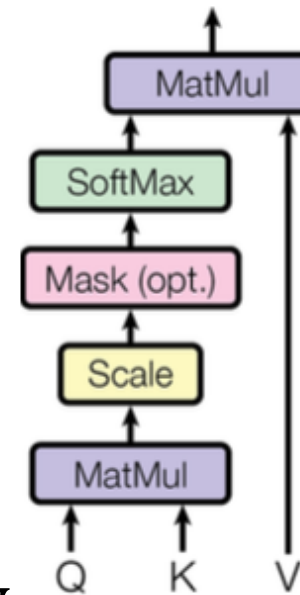
Transformers

From an input embedding three roles are played:

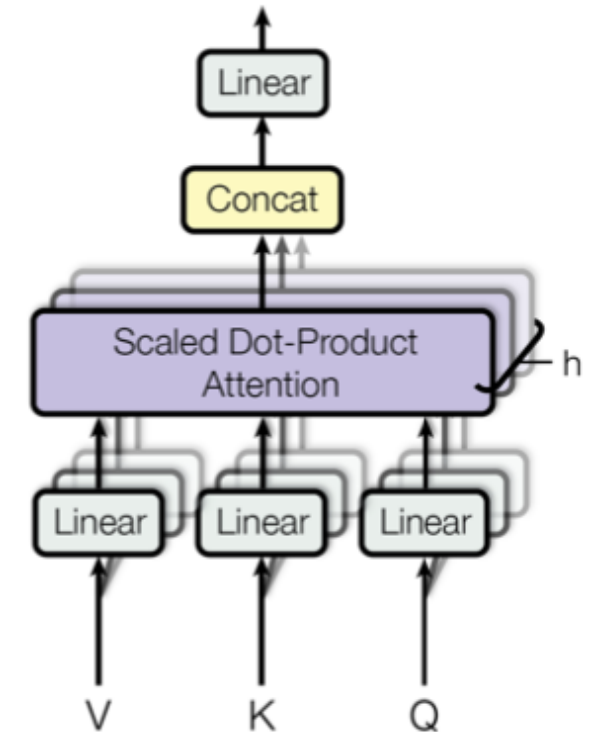
- Query → the current focus of attention
- Key → a preceding input
- Value → the value for the computation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right) V$$

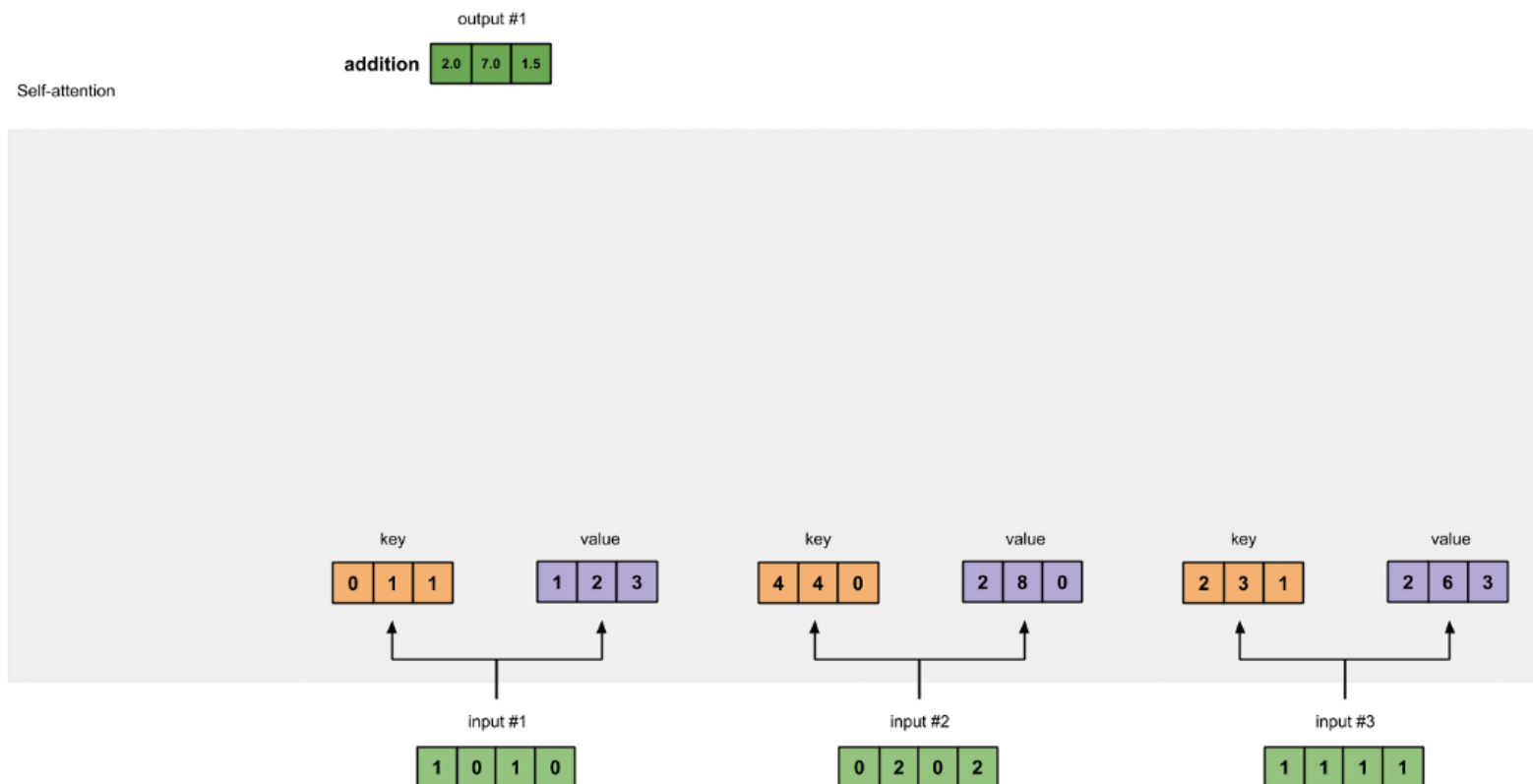
Scaled Dot-Product Attention



Multi-Head Attention



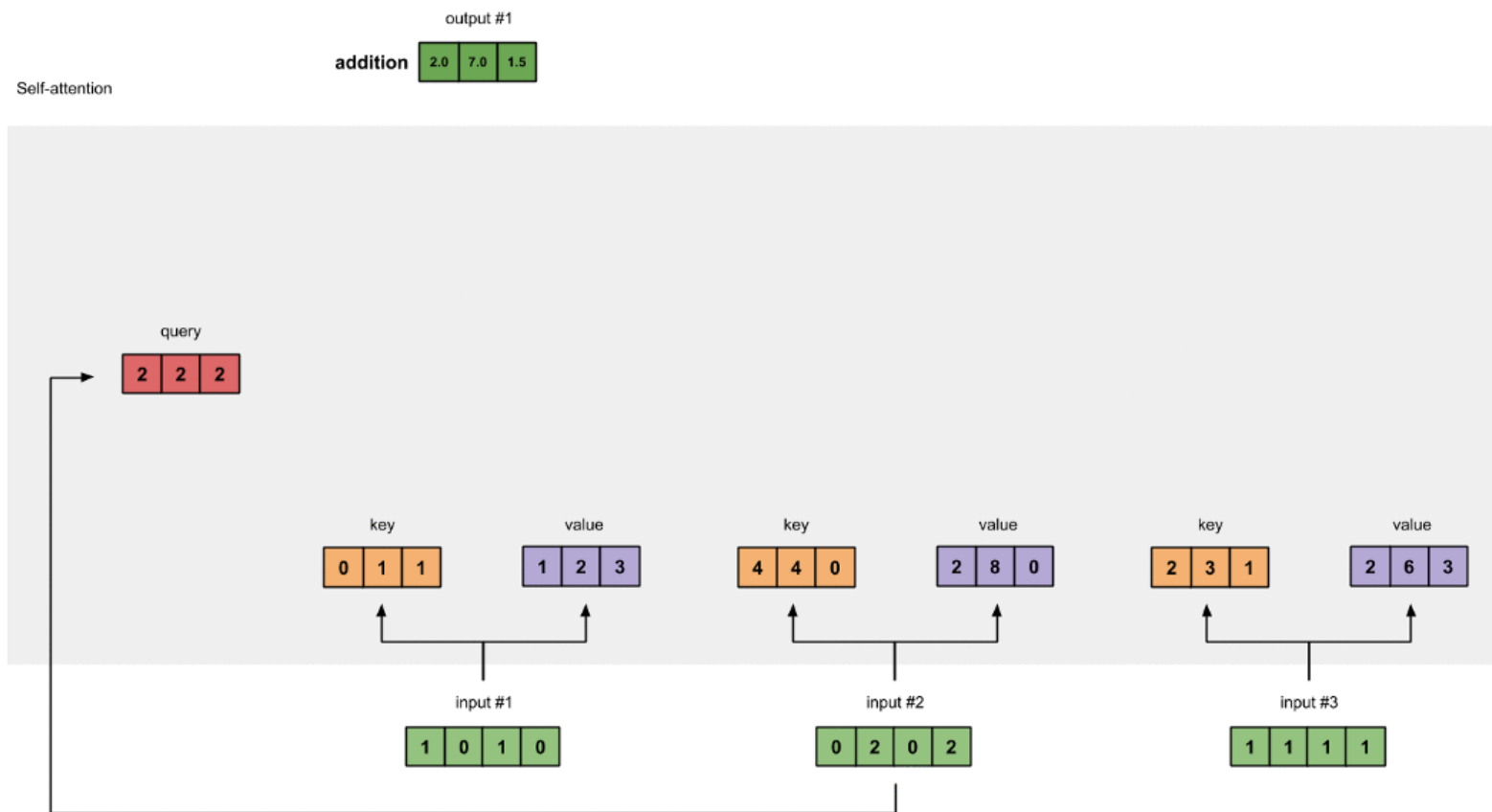
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

From a given input,
calculate
Key, Value

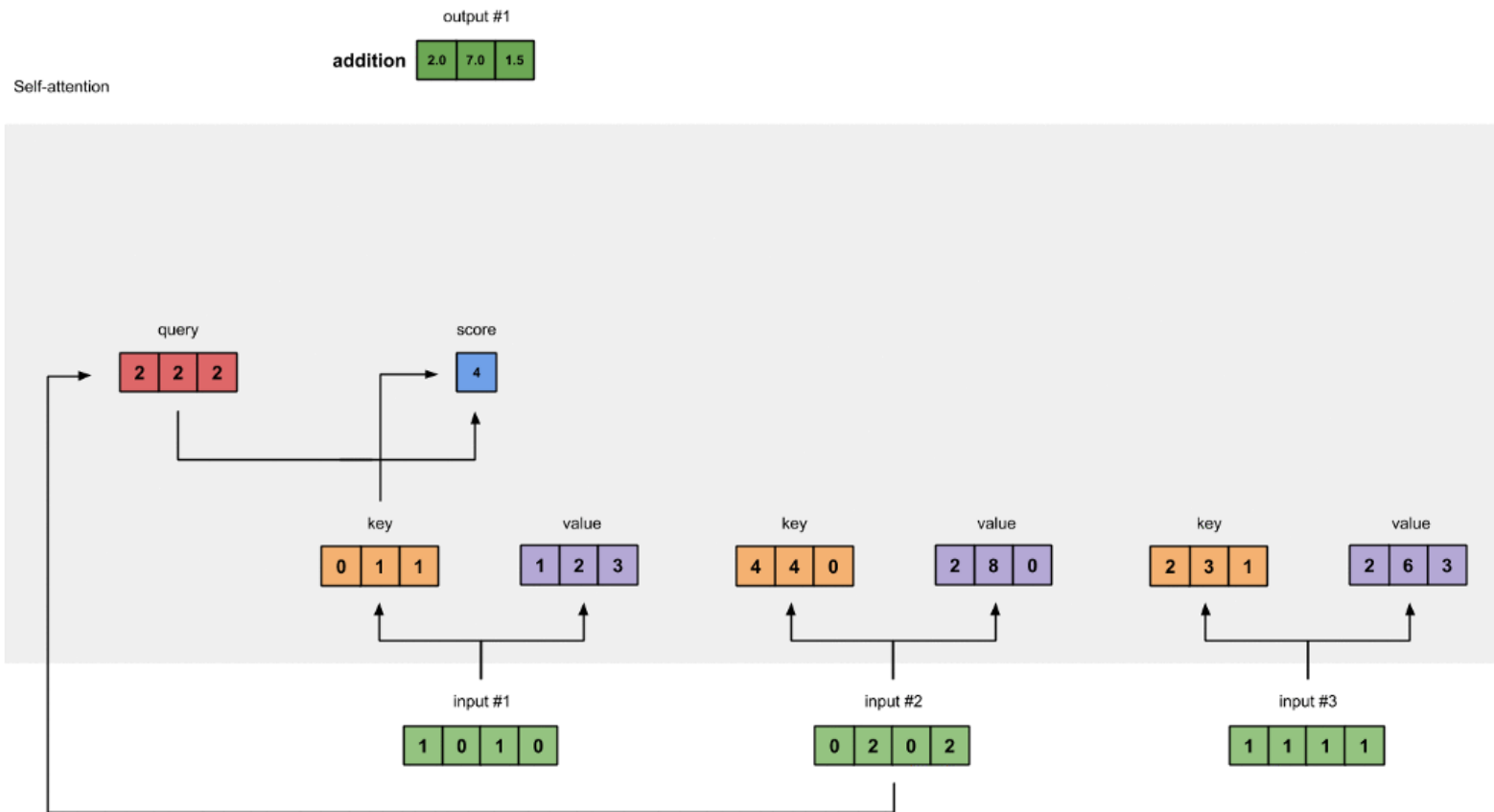
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Calculate the
Query of the given
input

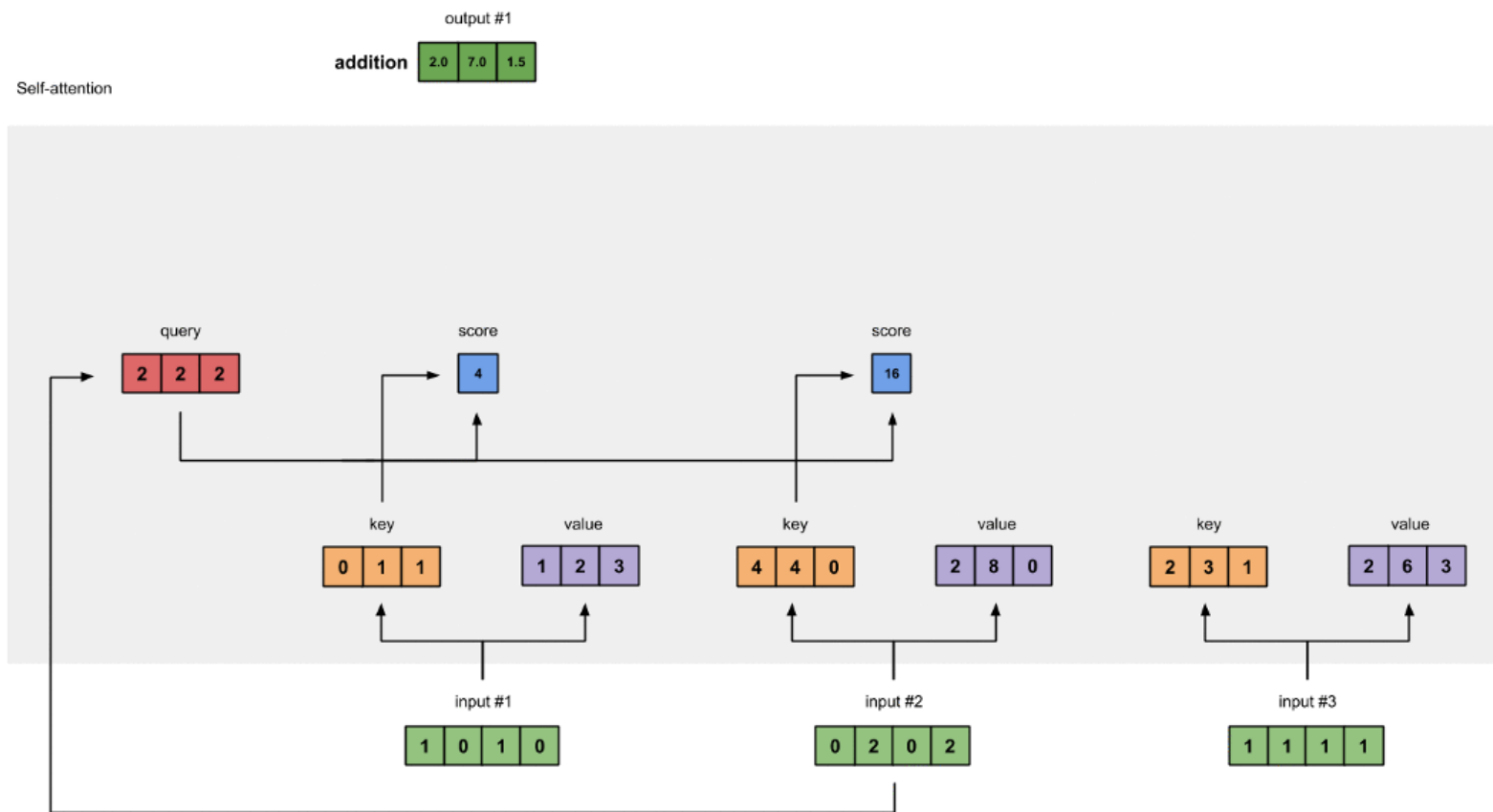
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Calculate score as
Query x Key, over
all the keys in the
sequence

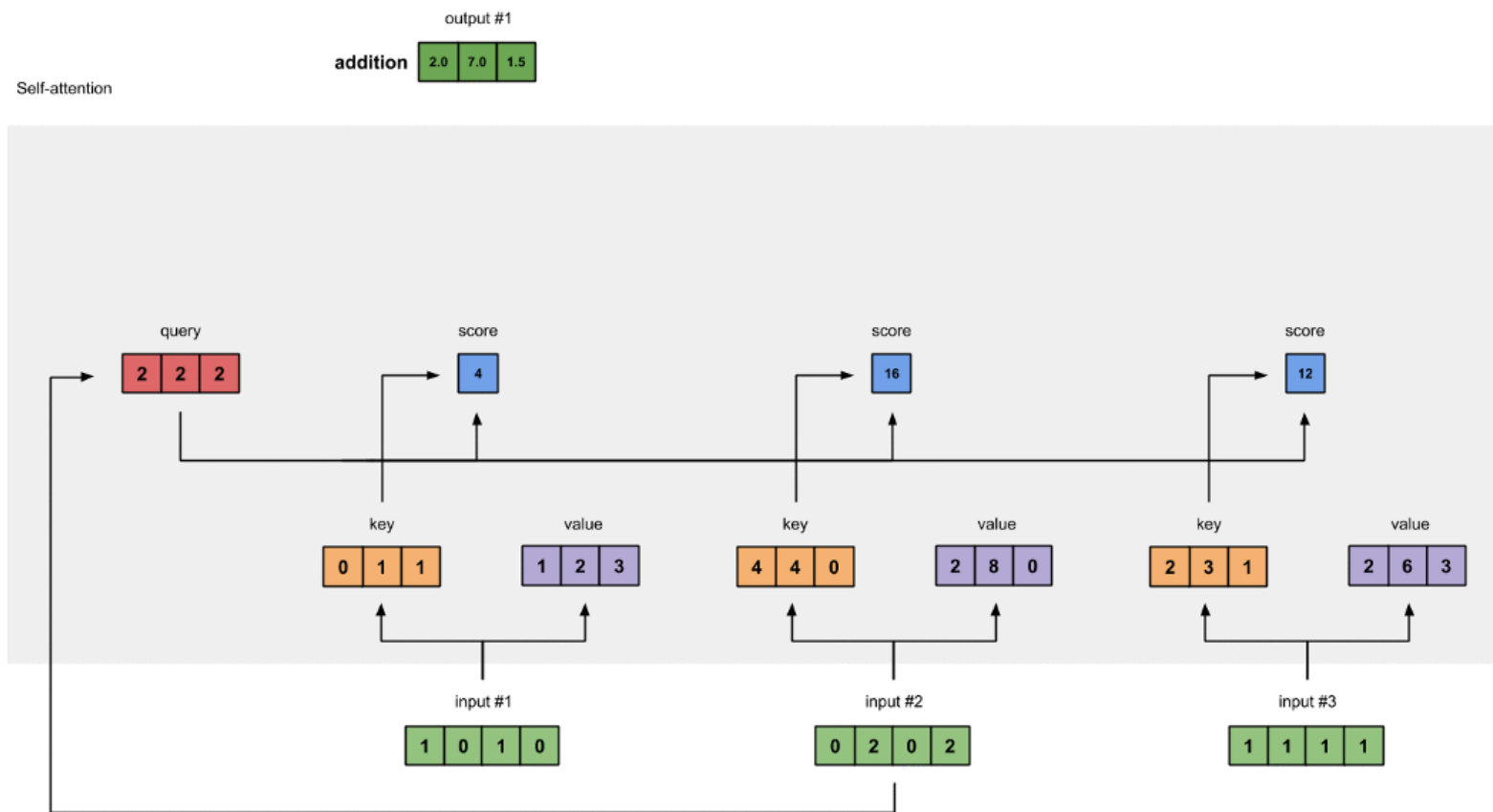
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Calculate score as
Query x Key, over
all the keys in the
sequence

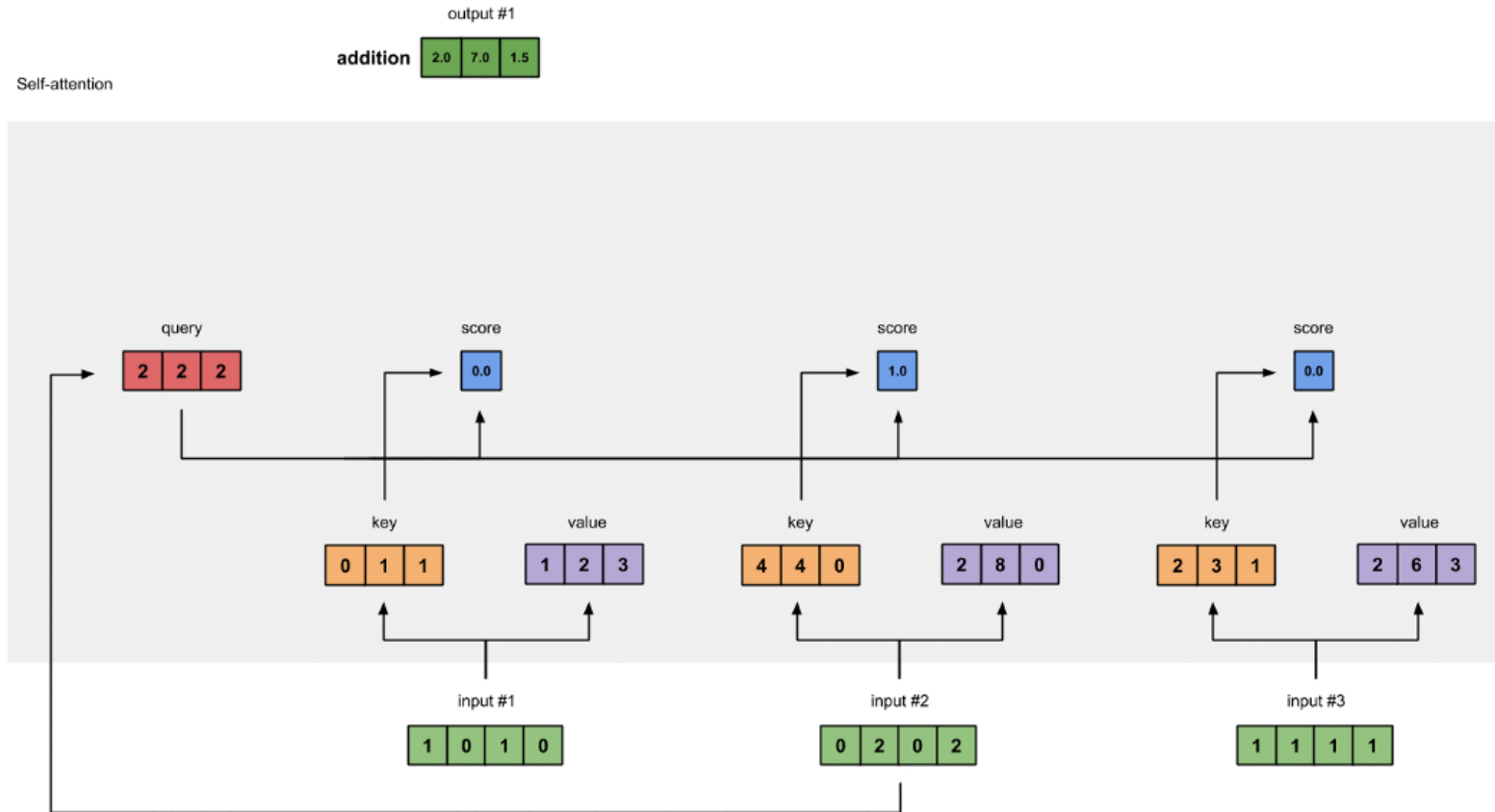
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Calculate score as
Query x Key, over
alla the keys in the
sequence

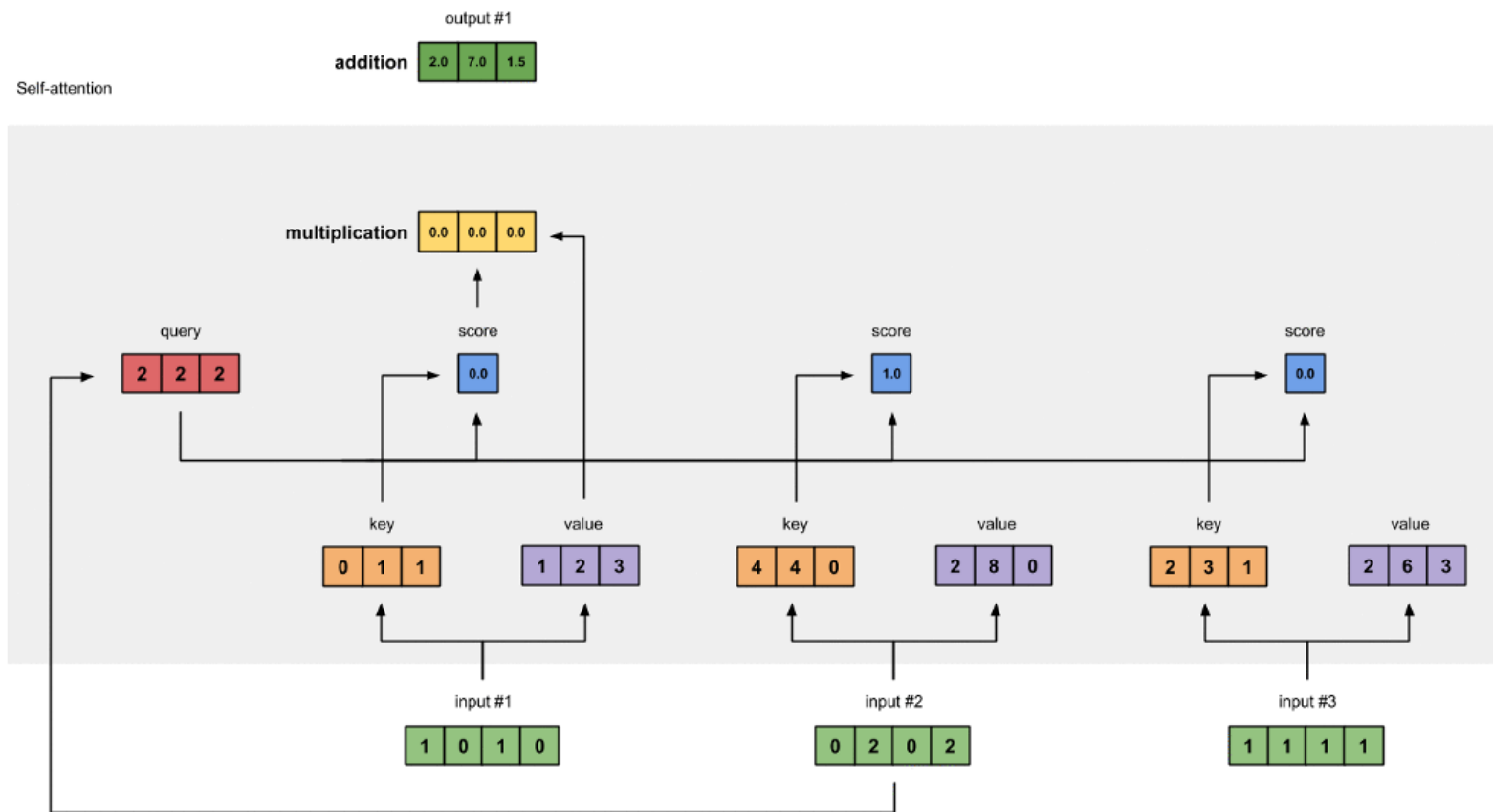
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Apply softmax and transform the previous obtained scores in probabilities

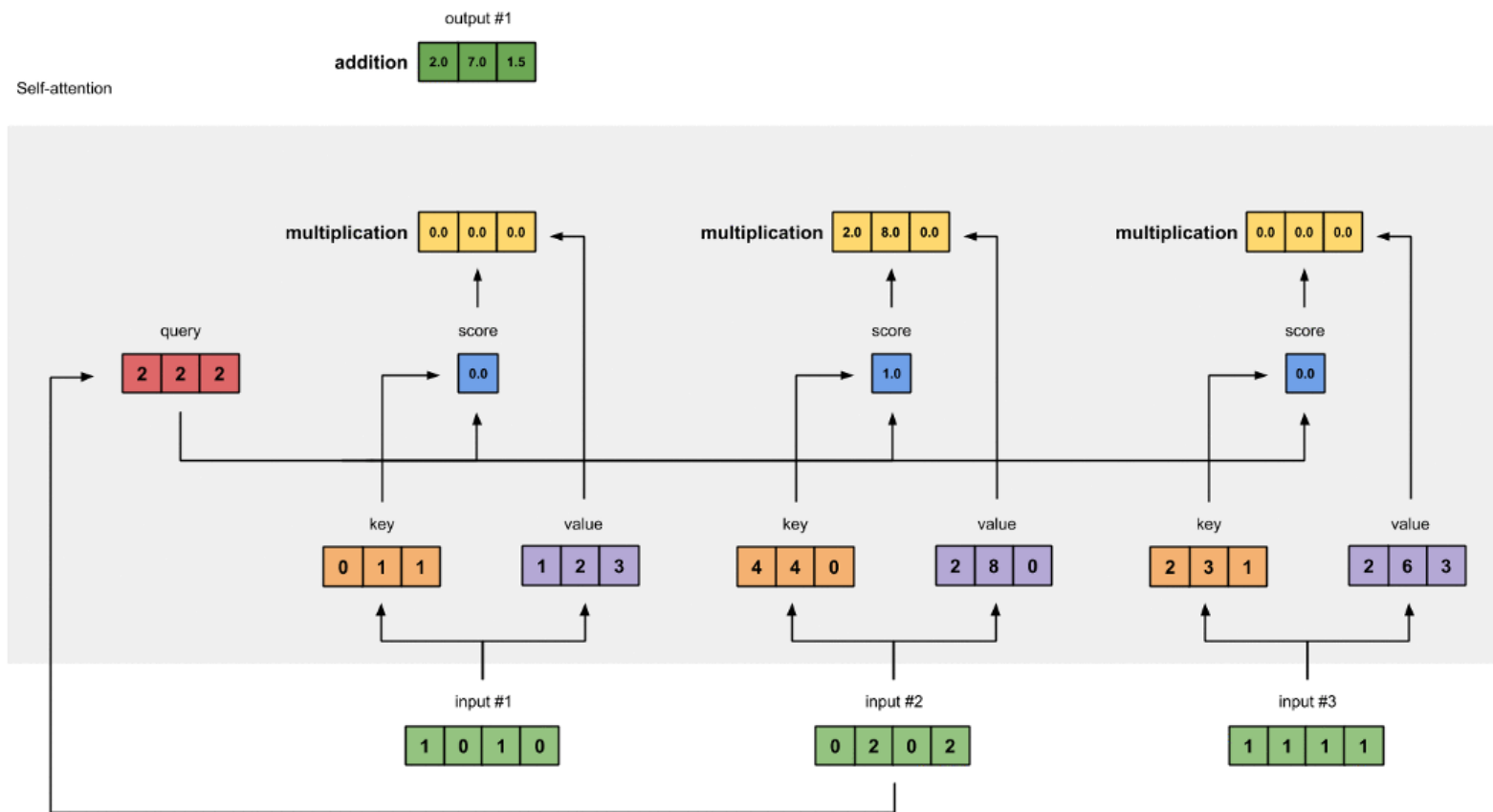
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Multiply the obtained score for the Value to obtain the attention value associated to the input for the current Query

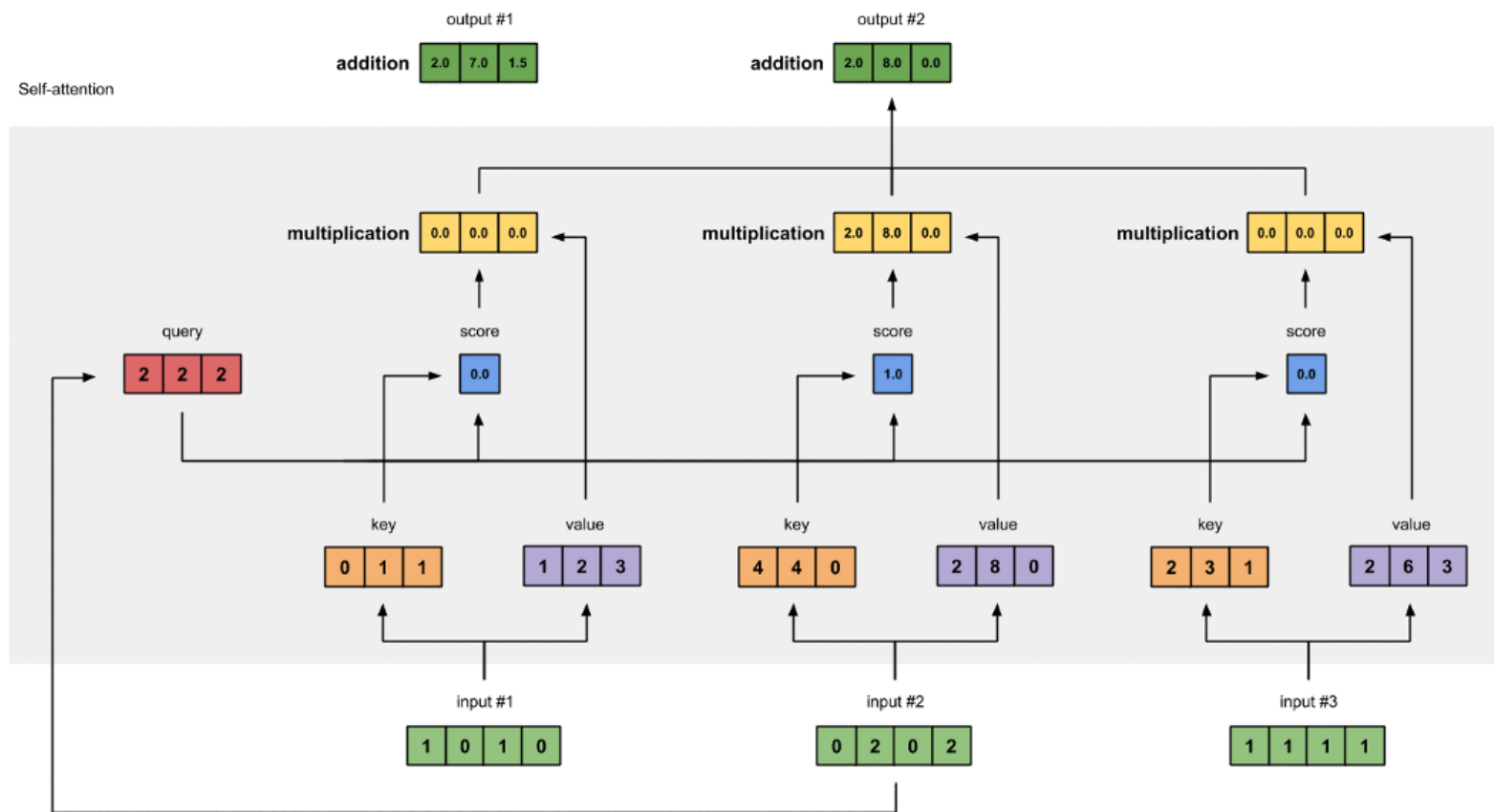
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Multiply the obtained score for the Value to obtain the attention value associated to the input for the current Query

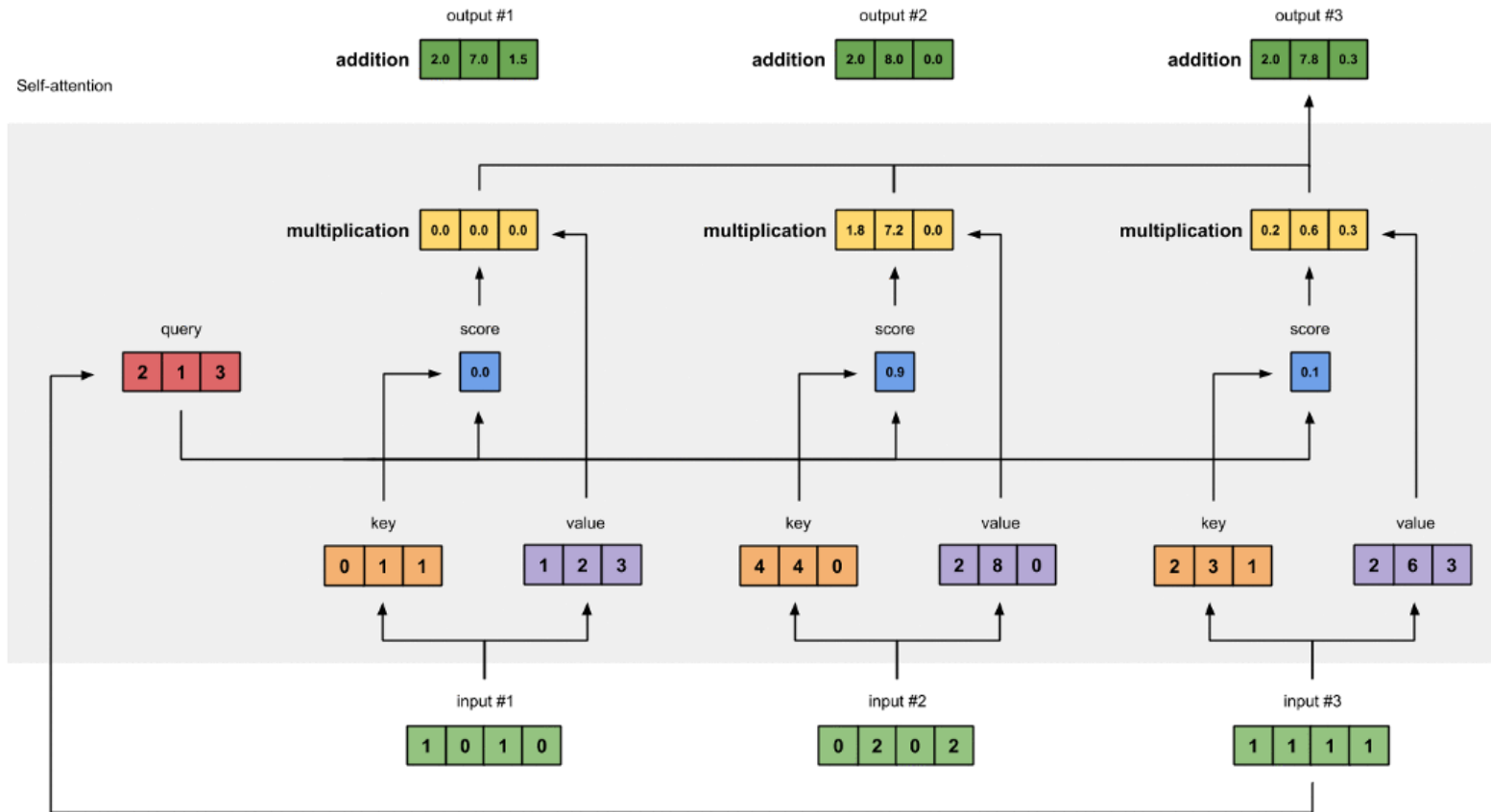
Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

The output is the weighted sum of the obtained attention scores

Transformers



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{(\sqrt{d_k})}\right)V$$

Iterate the process for all inputs in the sequence.
It is the self-attention mechanisms and I can see from left-to-right and from right-to-left

Transformers

